Software Delivery with OpenShift Pipelines and GitOps

Version 1, 2026-10-26

Introduction

Welcome to this quick course on the ***Software Delivery with OpenShift Pipelines and GitOps***. This course is the `third` in a series of `six` courses about **Application Platform Delivery**:

- *Introduction to Application Platforms and Platform Engineering*
- *Software Factory Foundations with OpenShift Dev Spaces*
- *Software Delivery with OpenShift Pipelines and GitOps* (*This course*)
- *DevEx with Red Hat Developer Hub*
- *Application Modernization with MTA*
- *Trusted Software Supply Chain (TSSC)* (Coming Soon)

What's in this course?

In this training, you'll explore the role of platform services, Continuous Integration (CI) with **Red Hat OpenShift Pipelines**, Continuous Delivery (CD) with **Red Hat OpenShift GitOps**, and application promotion workflows. Through hands-on exercises, you'll build, test, deploy, promote, and roll back applications across environments using automated, GitOps-driven workflows.

By the end of the training, you'll understand how these capabilities work together to create a repeatable software delivery process that improves developer productivity, deployment consistency, security, and application delivery reliability.

Duration: 4.5 hours

Contributors

The PTL team acknowledges the valuable contributions of the following Red Hat associates:

- Andrew Block (SME)
- Bill Wheatley (SME)
- Bryan Saunders (SME)
- Evan Shortiss (SME)
- Julio Villarreal Pelegrino (SME)
- Kristopher Godek (SME)
- Roman Martin Gil (SME)
- Senthil Kumar (SME)
- Sergio Canales Espinoza (SME)
- Yogita Soni (Content Architect)
- Lisa Edgell (Learning Product Manager)
- Caitlyn Rymarchyk (Learning Product Manager)

- Anna Swicegood (Learning Product Manager)

Objectives

On completing this course, you should be able to:

- Create and automate CI pipelines using OpenShift Pipelines and Tekton.
- Build and publish container images using automated workflows.
- Configure OpenShift GitOps and Argo CD for continuous deployment.
- Manage application deployment configuration using Git as the source of truth.
- Connect CI pipelines with GitOps workflows to automate application delivery.
- Promote application images across development and staging environments.
- Implement GitOps-based deployment and promotion workflows.
- Monitor application deployments and verify observability and deployment status.
- Apply rollback strategies to restore applications to a previous version.
- Build a repeatable, automated path from source code to deployed and continuously managed applications.

Prerequisites

This course assumes that you have the following prior experience:

1. Red Hat OpenShift Technical Overview (DO080) v4.20 or Kubernetes Basics

Lab Environment

No new lab environment is required for this module. Continue using the **same RHDP lab environment** that you provisioned in ***Software Factory Foundations with OpenShift Dev Spaces***.

The environment includes the **"Service Enablement: App Platform Software Factory"** and **"Service Enablement: App Platform Runtime"** labs required for this module.

Do **not** provision, delete, reset, or reprovision the environment. The labs in this module build on the applications, configurations, and platform resources created in ***Software Factory Foundations with OpenShift Dev Spaces***.

The cluster URLs and credentials are available from the **My Services** page in RHDP.

OpenShift Clusters

Environment	Purpose	Used In
Service Enablement: App Platform Software Factory (Primary Cluster)	Development platform where you'll build the internal developer platform, configure CI/CD, and deploy applications.	• *Software Factory Foundations with OpenShift Dev Spaces* • *Software Delivery with OpenShift Pipelines and GitOps* • *DevEx with Red Hat Developer Hub* • *Application Modernization with MTA*
Service Enablement: App Platform Runtime (Secondary Cluster)	Runtime environment used to demonstrate GitOps-based promotion between environments.	• *Software Delivery with OpenShift Pipelines and GitOps*

The required namespaces have already been created and are ready for use.

Internal Lab Account Credentials

Use the following accounts throughout the learning path.

Username	Password	Purpose
`platform-admin`	`openshift`	Platform administrator
`platform-developer`	`openshift`	Application developer
`platform-user`	`openshift`	Standard user

Unless instructed otherwise, authenticate to GitLab, Quay, Vault, and other integrated services using

Keycloak Single Sign-On (SSO).

Platform Services

The following services are pre-configured and available throughout the labs.

Service	Purpose
Red Hat Build of Keycloak	Provides centralized authentication and Single Sign-On (SSO).
GitLab	Hosts the application source code and Git repositories.
Red Hat Quay	Stores container images built during the labs.
OpenShift GitOps	Continuously synchronizes applications and platform resources from Git.
HashiCorp Vault	Securely stores secrets used by applications and CI/CD pipelines.

All service URLs are available from the **Red Hat Demo Platform (RHDP)** **Lab Information** in **My Services** section page.

CI with OpenShift Pipelines



What's in this section?

In this section, you'll learn how **Red Hat OpenShift Pipelines** enables automated Continuous Integration (CI) and container image builds. You'll explore how Tekton pipelines build and validate applications, create container images, and automate CI workflows triggered by changes to the application source code.

Through hands-on labs, you'll create, test, and automate a complete CI pipeline that builds the application and publishes the resulting container image to the OpenShift internal registry.

OpenShift Pipelines Overview

In the previous module, you learned how a **Software Factory** provides standardized services, automation, and engineering practices that help teams deliver software consistently.

A key capability of a Software Factory is **Continuous Integration and Continuous Delivery (CI/CD)**.

Red Hat OpenShift Pipelines provides a cloud-native CI/CD framework for automating application build, test, validation, and delivery workflows directly on OpenShift.

OpenShift Pipelines is based on **Tekton** and uses Kubernetes-native resources to define repeatable software delivery processes.



Think about the steps your team performs every time application code changes. How many of those steps could be automated and executed consistently by a pipeline?

The Challenge

Without automated pipelines, application delivery often depends on developers or operations teams performing a sequence of manual steps.

A typical application delivery process may include:

- Building the application.
- Running unit and integration tests.

- Performing code quality and security checks.
- Building a container image.
- Pushing the image to a container registry.
- Deploying the application.
- Verifying the deployment.

When these activities are performed manually, teams can encounter inconsistent processes, human errors, and delays between development and deployment.

Different teams may also implement the same process differently, making software delivery difficult to standardize and scale.



The more manual steps involved in software delivery, the greater the opportunity for inconsistency, errors, and delays.

OpenShift Pipelines addresses this challenge by allowing the complete workflow to be defined as **code** and executed automatically.

What is OpenShift Pipelines?

OpenShift Pipelines is a **cloud-native CI/CD solution** based on the open source Tekton project.

It runs pipeline workloads as Kubernetes-native resources on OpenShift.

Instead of implementing a delivery process through manually executed commands, teams define the required activities as reusable pipeline components.

For example:

Tasks

Define individual units of work such as compiling source code, running tests, building an image, or publishing an artifact.

Pipelines

Connect multiple tasks together to define the complete application delivery workflow and the order in which tasks execute.

Triggers

Automatically start pipeline runs in response to events such as source-code changes, pull requests, or other configured events.

Each pipeline execution creates a **PipelineRun** that represents a specific execution of the pipeline.

This allows software delivery to become repeatable, observable, and automated.

How OpenShift Pipelines Works

OpenShift Pipelines uses a task-based approach to software delivery.

Each task performs a specific operation, and tasks can be connected together to create an end-to-end workflow.

For example, an application pipeline might look like this:

1. Developer commits application source code.
2. A pipeline is triggered by the change.
3. The source code is retrieved from Git.
4. The application is compiled and packaged.
5. Automated tests are executed.
6. A container image is built.
7. The image is pushed to a container registry.
8. The application is prepared for deployment.

Each step is executed as part of the pipeline rather than requiring the developer to perform the operation manually.

This provides a consistent process that can be executed repeatedly for every application change.

Tasks and Pipelines

A fundamental concept in OpenShift Pipelines is the separation of individual activities into **tasks**.

A task represents a unit of work.

For example, a task could:

- Clone a Git repository.
- Run Maven or Gradle commands.
- Execute unit tests.
- Build a container image.
- Push an image to a registry.
- Run security checks.
- Update deployment configuration.

A **Pipeline** combines these tasks into a complete workflow.

The pipeline can define dependencies between tasks so that activities execute in the required order.

For example:

```
clone-source
|
v
build-application
|
v
run-tests
|
v
build-image
|
v
push-image
```

This task-based model makes complex delivery processes easier to understand, reuse, and maintain.

Cloud-Native Execution

Unlike traditional CI/CD systems that may rely on long-running build servers, OpenShift Pipelines executes tasks using Kubernetes-native resources.

Each task runs in its own container-based execution environment.

This provides several advantages:

Capability	**Benefit**
Containerized tasks	Each task runs in an isolated and reproducible environment.
Kubernetes-native	Pipeline resources are managed using the same platform used to run applications.
Scalability	Pipeline workloads can use OpenShift resources when required and scale with the platform.
Repeatability	The same task definitions can be executed consistently across pipeline runs.
Reusability	Common tasks can be reused across multiple application pipelines.

Because the pipeline runs on OpenShift, developers do not need to manually prepare a separate build environment for every application.

Pipelines as Code

One of the key principles of OpenShift Pipelines is **everything-as-code**.

Pipeline definitions, tasks, parameters, and configuration can be stored in Git alongside the application source code.

This makes the software delivery process:

- Version controlled.
- Reviewable.
- Reproducible.
- Auditable.
- Easy to modify.

For example, instead of documenting a manual build process in a separate document, the process itself can be represented through pipeline and task definitions.

When the delivery process changes, the change can be reviewed and tracked just like an application code change.



Treating the delivery process as code allows teams to version, review, reuse, and continuously improve their CI/CD automation.

From Code Change to Application Delivery

OpenShift Pipelines can automate the path from source code to a deployable application artifact.

A typical workflow looks like this:

1. A developer pushes a code change to Git.
2. The pipeline is triggered.
3. OpenShift Pipelines retrieves the source code.
4. The application is built.
5. Automated tests validate the application.
6. Additional quality or security checks are performed.
7. A container image is created.
8. The image is published to a container registry.
9. The resulting artifact is made available for the next stage of the delivery process.

This creates a repeatable software delivery process where the same sequence of activities is executed for every application change.

Developers can focus on writing application code while OpenShift Pipelines automates the surrounding build and validation activities.

Why OpenShift Pipelines?

OpenShift Pipelines helps organizations move from manually executed delivery processes to standardized, automated workflows.

For developers:

- Automate repetitive build and test activities.
- Get consistent results for every code change.
- Reduce manual steps in application delivery.
- Receive faster feedback from automated validation.

For Platform Engineers:

- Standardize CI/CD workflows across development teams.
- Provide reusable pipeline tasks.
- Manage CI/CD using Kubernetes-native resources.
- Integrate pipelines with other OpenShift services.

For organizations:

- Improve software delivery consistency.
- Reduce human error.
- Increase delivery automation.
- Improve traceability of application changes.
- Create repeatable and scalable CI/CD processes.

OpenShift Pipelines in the Software Factory

Within a **Software Factory**, OpenShift Pipelines provides the automation engine that moves application changes through the software delivery process.

A simplified Software Factory workflow looks like this:

1. Developers develop and test applications in a standardized development environment.
2. Developers commit application changes to Git.
3. OpenShift Pipelines is triggered by the change.
4. The pipeline builds and validates the application.
5. The pipeline creates and publishes the application artifact.
6. The resulting artifact is passed to the next stage of the software delivery process.

This allows developers to focus on application development while the Software Factory automates the repetitive activities required to build and validate software.



OpenShift Pipelines does not simply execute a sequence of commands. It provides a Kubernetes-native framework for defining, executing, and managing repeatable software delivery workflows.

Benefits

OpenShift Pipelines provides several benefits for modern application delivery:

- **Automation:** Eliminates repetitive manual build and delivery activities.
- **Consistency:** Executes the same workflow for every pipeline run.
- **Reusability:** Allows common tasks and pipeline patterns to be reused.
- **Isolation:** Executes individual tasks in containerized environments.
- **Scalability:** Uses the OpenShift platform to execute pipeline workloads.
- **Traceability:** Provides visibility into individual tasks and pipeline executions.
- **Everything-as-code:** Allows CI/CD workflows to be stored and versioned alongside application code.

Reflection

▼ Think about your own organization (Click to expand)



- Which application delivery steps are currently performed manually?
- Which steps could be converted into reusable pipeline tasks?
- How consistent is the build and test process across development teams?
- Which CI/CD configurations should be stored and versioned in Git?
- How could automated pipelines improve feedback to developers?

```
<div class="key-card">
```

Key Takeaway

OpenShift Pipelines provides a Kubernetes-native CI/CD framework based on Tekton that automates application build, test, validation, and delivery workflows. By defining delivery processes as reusable, version-controlled tasks and pipelines, organizations can create consistent, repeatable, and scalable software delivery processes within the Software Factory.

```
</div>
```

Container Image Builds

In the previous section, you learned how **OpenShift Pipelines** provides a cloud-native framework for automating software delivery workflows.

A pipeline can orchestrate activities such as building source code, running tests, creating container images, and publishing application artifacts.

However, a pipeline itself does not define how a container image is constructed.

OpenShift provides several approaches for **building container images** from application source code.



Think about what happens after a developer commits application code. How is that source code transformed into a container image that can be deployed?

The Challenge

Developers need a reliable way to transform application source code into container images.

A traditional container build process might require developers to:

- Create and maintain a Dockerfile.
- Install and configure container build tools.
- Build the image.
- Tag the image.
- Push the image to a container registry.
- Manage build environments and dependencies.
- Keep the build process consistent across teams.

When each team manages its own image-building process, organizations can end up with different tools, build environments, and practices.

OpenShift provides native and Kubernetes-based approaches that allow image builds to be integrated into the application delivery platform.



OpenShift can provide the build environment, build strategy, and automation required to transform application source code into a container image without requiring developers to manage the underlying build infrastructure themselves.

Building Images in the Software Factory

Within a **Software Factory**, container image builds are an important step between application development and application deployment.

A simplified workflow looks like this:

1. Developers commit application source code to Git.
2. A CI pipeline is triggered.
3. The application is compiled and tested.
4. The application source is provided to an image build process.
5. A container image is created.
6. The image is pushed to an image registry.
7. The image becomes available for deployment.

OpenShift provides different mechanisms for performing the image-building step.

BuildConfig

One traditional approach to building container images on OpenShift is the **BuildConfig**.

A BuildConfig is a declarative OpenShift resource that defines how an image build should be performed.

It describes important aspects of the build, including:

Configuration	Purpose
Source	Defines where the application source code or build input comes from.
Strategy	Defines how OpenShift should build the container image.
Output	Defines where the resulting image should be published.
Triggers	Defines events that can automatically start a new build.

The BuildConfig provides a consistent, declarative definition of the image-building process.

Instead of manually executing container build commands, developers and platform teams can define the build process as an OpenShift resource.

Build Strategies

OpenShift supports several build strategies for creating container images.

The most common strategies are:

Source-to-Image (S2I)

S2I combines application source code with a builder image that contains the required runtime and build tools. This allows developers to build application images without maintaining a Dockerfile for every application.

Docker Build

Uses a Dockerfile to define how the container image should be constructed. This provides developers with direct control over the image build instructions.

Custom Build

Allows advanced build logic by using a custom builder image. This approach can be used when standard build strategies do not meet the application's requirements.

The appropriate strategy depends on the application, the level of customization required, and the organization's container build standards.

Source-to-Image

Source-to-Image (S2I) is designed to make application image creation simpler for developers.

Instead of requiring developers to create a Dockerfile, S2I combines application source code with a preconfigured builder image.

For example, a Java application can be provided to a Java S2I builder image.

The builder image contains the tools and logic required to:

- Install application dependencies.
- Compile the application.
- Package the application.
- Configure the runtime environment.
- Produce a runnable container image.

This allows developers to focus on application source code while the platform provides the underlying build environment.

Docker Builds

Docker builds provide a more explicit approach to image construction.

The application repository contains a **Dockerfile** that defines the instructions used to construct the image.

For example, a Dockerfile can define:

- The base image.
- Required packages and dependencies.
- Application files.
- Environment variables.
- Startup commands.
- Exposed ports.

This approach provides greater control over the resulting image but requires developers to maintain the Dockerfile and understand container image construction.

Custom Builds

Some applications require build processes that do not fit standard S2I or Docker-based approaches.

Custom builds allow organizations to provide their own builder image and build logic.

This can be useful when:

- Specialized build tools are required.
- Existing build processes need to be preserved.
- Custom image construction logic is required.
- Standard build strategies do not provide sufficient flexibility.

Custom builds provide flexibility, but they also introduce additional complexity and maintenance requirements.

Modern Image Builds with Shipwright

As Kubernetes-native application development has evolved, newer approaches have emerged for building container images.

Shipwright provides a Kubernetes-native framework for defining and managing container image builds.

Rather than relying exclusively on a single build mechanism, Shipwright provides a framework that can work with different build strategies and tools.

For example, image builds can use technologies such as:

- Buildah.
- Kaniko.
- Cloud Native Buildpacks.

This provides greater flexibility when standardizing image builds across different types of applications.

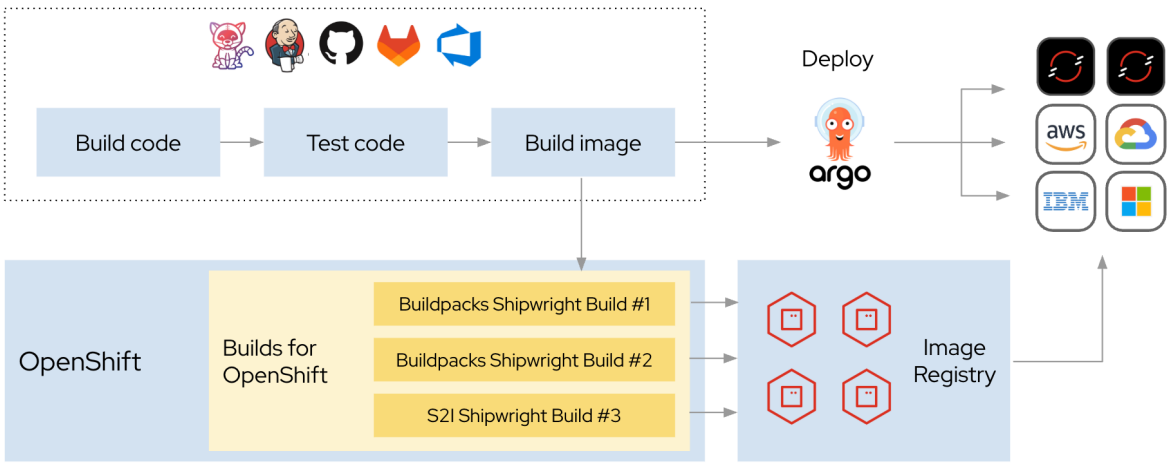


Figure 1. OpenShift image build workflow(Click the image to view it in full size in a new browser tab)

The diagram illustrates how an application delivery workflow can move from source code through build and test activities to container image creation and publication.

OpenShift provides the platform where these build workloads execute, while the resulting images are stored in an image registry and can then be consumed by deployment processes.

BuildConfig and Shipwright

BuildConfig and Shipwright address the same broader requirement—**building container images**—but use different approaches.

Capability	BuildConfig	Shipwright
Model	OpenShift-native build resource	Kubernetes-native build framework
Build definition	Defined through BuildConfig	Defined through Kubernetes resources
Build strategies	S2I, Docker, and Custom	Extensible strategy-based model
Build engines	Integrated OpenShift build mechanisms	Can integrate with different build technologies
Use case	Traditional OpenShift image builds	Flexible and extensible Kubernetes-native image builds

BuildConfig remains an important OpenShift build mechanism, while Shipwright provides an extensible approach for organizations that need greater flexibility in how container images are constructed.

Image Registry

Once an image has been built, it needs to be stored somewhere that downstream systems can access.

An **image registry** stores container images and makes them available for deployment.

A typical workflow is:

```
Application Source
       |
       v
    Build
       |
       v
Container Image
       |
       v
Image Registry
       |
       v
Application Deployment
```

The registry acts as the handoff point between the image-building process and the deployment process.

A Software Factory can integrate OpenShift image builds with an organization-approved container registry to provide a consistent path from source code to deployable artifacts.

Image Builds and OpenShift Pipelines

OpenShift Pipelines and image-building technologies have complementary responsibilities.

OpenShift Pipelines **orchestrates the workflow**, while the image build mechanism **performs the image construction**.

For example, a pipeline might execute the following sequence:

1. Clone the application source code.
2. Build the application.
3. Run automated tests.
4. Start the container image build.
5. Push the resulting image to a registry.
6. Continue with the next stage of the delivery process.

This separation allows the same pipeline model to work with different image-building technologies.



Think of OpenShift Pipelines as the **orchestrator** and the image build system as the **engine that creates the container image**. The pipeline determines when and in what order the build occurs, while the build mechanism determines how the image is constructed.

Why Container Image Builds Matter

Container image builds are a critical part of cloud-native application delivery.

A reliable image build process provides:

- **Consistency:** The same build process can be repeated for every application change.
- **Automation:** Image creation can be triggered automatically by CI/CD workflows.
- **Reproducibility:** Build definitions can be stored and versioned as code.
- **Security:** Organizations can standardize approved base images and build processes.
- **Integration:** Images can be automatically published to registries for downstream deployment.
- **Scalability:** Build workloads can run on the OpenShift platform instead of individual developer workstations.

Container Image Builds in the Software Factory

Putting the pieces together, a Software Factory can provide an automated path from source code to a deployable container image.

A typical workflow looks like this:

1. Developer commits application code.
2. OpenShift Pipelines starts the CI workflow.
3. The application is built.
4. Automated tests are executed.
5. OpenShift performs the container image build.
6. The resulting image is pushed to an image registry.
7. The image is made available for the deployment process.

This creates a repeatable path from **application source code to a deployable container artifact**.



Container image builds are the bridge between application source code and the container artifacts used by cloud-native platforms. OpenShift provides multiple approaches so organizations can select the build mechanism that best fits their applications and delivery practices.

Benefits

For developers:

- Reduce the need to manage local container build environments.
- Use standardized build processes.
- Focus on application source code rather than build infrastructure.
- Automatically create container images through CI/CD workflows.

For Platform Engineers:

- Standardize image-building practices.
- Provide reusable build strategies.
- Integrate image builds with CI/CD pipelines.
- Centralize build infrastructure on OpenShift.
- Support different application build requirements.

For organizations:

- Create repeatable image-building processes.
- Improve software delivery consistency.
- Reduce manual image management.
- Establish standardized container build practices.
- Integrate image creation into the Software Factory.

Reflection

▼ **Think about your own organization (Click to expand)**

[ROOT\:mini illustration lightbulb red RH.Small\ square\ transparent] | *ROOT\:mini-illustration-lightbulb-red-RH.Small_square_transparent.png*

- How are container images built in your organization today?
- Do developers need to maintain Dockerfiles for every application?
- Which applications could benefit from S2I or Buildpacks?
- Where are your container images stored?
- How could standardized image builds improve your CI/CD process?

```
<div class="key-card">
```

Key Takeaway

OpenShift provides multiple approaches for building container images, including traditional **BuildConfig** strategies such as S2I, Docker, and Custom builds, as well as newer Kubernetes-native approaches such as **Shipwright**. OpenShift Pipelines can orchestrate these build processes as part of an automated path from application source code to a deployable container image.

```
</div>
```

Lab 1: CI Pipeline

In this lab, you'll prepare the foundational resources required for Tekton pipelines.

You'll create a shared workspace for pipeline tasks and configure a ServiceAccount with the permissions required to build container images and deploy application resources.

By the end of this lab, you will have:

- Created a PersistentVolumeClaim for the pipeline workspace
- Created a ServiceAccount for running Tekton pipelines
- Granted the ServiceAccount permissions to manage resources in the development project

Create the Pipeline Workspace and Service Account

Tekton pipelines use a shared workspace to exchange files between tasks. In this exercise, you'll create a PersistentVolumeClaim (PVC) that provides persistent storage for the pipeline, along with a ServiceAccount that Tekton uses to execute pipeline tasks.

1. Open the **OpenShift Console URL** listed on the **RHDP Lab Environment** page and sign in using any one of the credentials listed in the **Info** page.
2. Click the >_ (**Web Terminal**) icon in the upper-right corner to launch a terminal session.

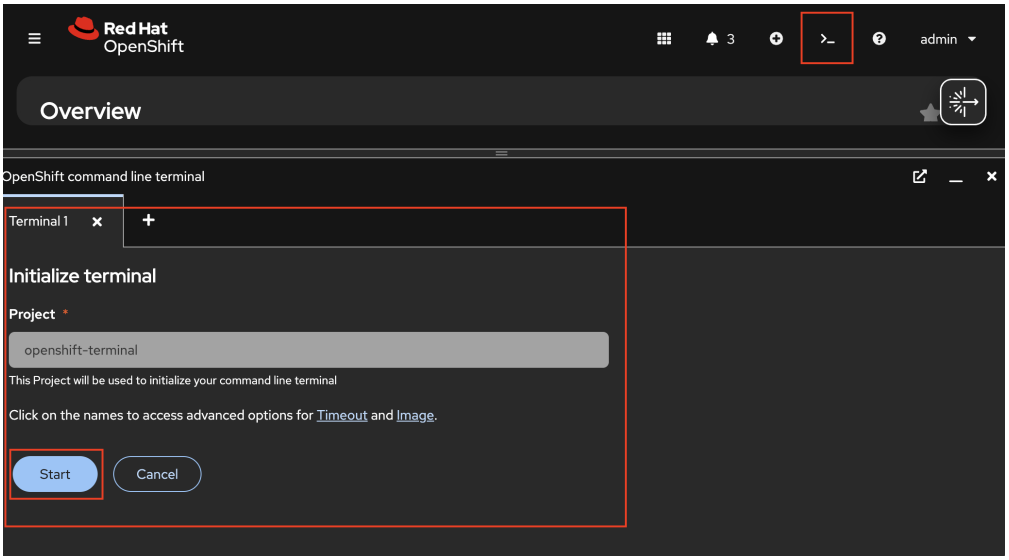


Figure 2. Web Terminal in Openshift console(Click the image to view it in full size in a new browser tab)

1. Create the **pipeline workspace** and **ServiceAccount** by running the following command:

```
oc apply -f - <<'EOF'
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: pipeline-workspace-pvc
  namespace: etx-app-dev
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 5Gi
---
apiVersion: v1
kind: ServiceAccount
metadata:
  name: pipeline
  namespace: etx-app-dev
EOF
```

The command creates:

- A `pipeline-workspace-pvc` PersistentVolumeClaim for sharing files between pipeline tasks.
- A `pipeline` ServiceAccount that Tekton uses to execute the pipeline.



If the `pipeline` ServiceAccount already exists (for example, it was created during namespace provisioning), you may see `serviceaccount/pipeline configured` instead of `created`. This is expected and can be safely ignored.

Grant Pipeline Permissions

The pipeline ServiceAccount requires permission to create and manage application resources within the development project.

1. Run the following command:

```
oc adm policy add-role-to-user edit \
  system:serviceaccount:etx-app-dev:pipeline \
  -n etx-app-dev
```

The `edit` role allows the pipeline to create and manage resources such as ImageStreams, Deployments, Services, Routes, and other application resources in the `etx-app-dev` project.

Verify the Pipeline Resources

Confirm that the workspace and ServiceAccount were created successfully.

```
oc get pvc pipeline-workspace-pvc -n etx-app-dev

oc get sa pipeline -n etx-app-dev
```

The PersistentVolumeClaim will show a `STATUS` of `Pending`. This is expected behavior because the storage class uses `WaitForFirstConsumer` volume binding mode, which delays provisioning until a Pod that uses the PVC is scheduled. The PVC will automatically transition to `Bound` when the pipeline runs in the next lab.

Lab 2: Create the CI Pipeline

In this lab, you'll create a Tekton CI pipeline that builds the application, retrieves container registry credentials from HashiCorp Vault, and produces a container image ready for deployment.

By the end of this lab, you will have:

- Created a custom Tekton Task to retrieve registry credentials from Vault
- Built a reusable Tekton Pipeline for application builds
- Configured the pipeline to clone source code, build the application, and build a container image

Create a Task to Retrieve Registry Credentials

The CI pipeline requires credentials to authenticate with the container registry before it can push application images.

Create a custom **Tekton Task** that authenticates to **HashiCorp Vault** using the Kubernetes authentication method configured in the previous supporting services lab, retrieves the registry credentials, and makes them available to subsequent pipeline tasks.

First, retrieve the Vault route from the OpenShift cluster.

```
VAULT_URL="https://$(oc get route \
  -n vault \
  -o jsonpath='{.items[0].spec.host}')"

echo "${VAULT_URL}"
```

Next, create the Tekton Task.

```
oc apply -f - <<EOF
apiVersion: tekton.dev/v1
kind: Task
metadata:
  name: vault-fetch-registry-creds
  namespace: etx-app-dev
spec:
  workspaces:
    - name: source
  results:
    - name: registry-server
    - name: registry-username
    - name: registry-password
  steps:
    - name: fetch
      image: registry.access.redhat.com/ubi9/ubi-minimal:latest
      script: |
        #!/bin/sh
        set -e

        # Install jq
        microdnf install -y jq && microdnf clean all

        # Authenticate to Vault using ServiceAccount token
        VAULT_ADDR="${VAULT_URL}"
        SA_TOKEN=\$(cat /var/run/secrets/kubernetes.io/serviceaccount/token)

        VAULT_TOKEN=\$(curl -s --request POST \
          "\${VAULT_ADDR}/v1/auth/kubernetes/login" \
          --data "{\"role\":\"pipeline\",\"jwt\":\"\${SA_TOKEN}\"}" \
          | jq -r '.auth.client_token')

        # Fetch registry credentials
        CREDS=\$(curl -s --header "X-Vault-Token: \${VAULT_TOKEN}" \
          "\${VAULT_ADDR}/v1/secret/data/registry")

        echo "\${CREDS}" | jq -r '.data.data.server' \
          | tr -d '\n' > \$(results.registry-server.path)
        echo "\${CREDS}" | jq -r '.data.data.username' \
```

```
          | tr -d '\n' > \$(results.registry-username.path)
        echo "\${CREDS}" | jq -r '.data.data.password' \
          | tr -d '\n' > \$(results.registry-password.path)
EOF
```



This Task authenticates to Vault using the ServiceAccount token mounted into the pipeline pod. It retrieves the registry server, username, and password from Vault and exposes them as Tekton Task results for use by downstream pipeline tasks.

Create the CI Pipeline

Next, create the Tekton Pipeline that orchestrates the complete CI workflow.

The pipeline performs the following activities:

- Clones the application source code from Git
- Builds and tests the application using Maven
- Retrieves registry credentials from Vault
- Builds the application container image using Buildah

Create the pipeline.

```
oc apply -f - <<'EOF'
apiVersion: tekton.dev/v1
kind: Pipeline
metadata:
  name: etx-app-base-app-ci
  namespace: etx-app-dev
spec:
  params:
    - name: git-url
      type: string
    - name: git-revision
      type: string
      default: main
    - name: image-name
      type: string
  workspaces:
    - name: shared-workspace
  tasks:
    - name: clone
      taskRef:
        resolver: cluster
        params:
          - name: kind
            value: task
          - name: name
            value: git-clone
```

```
        - name: namespace
          value: openshift-pipelines
    params:
      - name: URL
        value: $(params.git-url)
      - name: REVISION
        value: $(params.git-revision)
    workspaces:
      - name: output
        workspace: shared-workspace

  - name: build-and-test
    runAfter: [clone]
    taskRef:
      resolver: cluster
      params:
        - name: kind
          value: task
        - name: name
          value: maven
        - name: namespace
          value: openshift-pipelines
    params:
      - name: GOALS
        value: ["package", "-DskipTests=false"]
    workspaces:
      - name: source
        workspace: shared-workspace

  - name: fetch-creds
    runAfter: [clone]
    taskRef:
      name: vault-fetch-registry-creds
    workspaces:
      - name: source
        workspace: shared-workspace

  - name: build-image
    runAfter: [build-and-test, fetch-creds]
    taskRef:
      resolver: cluster
      params:
        - name: kind
          value: task
        - name: name
          value: buildah
        - name: namespace
          value: openshift-pipelines
    params:
      - name: IMAGE
        value: $(params.image-name):$(params.git-revision)
```

```
        - name: DOCKERFILE
          value: Containerfile
      workspaces:
        - name: source
          workspace: shared-workspace
EOF
```



The `git-clone`, `maven`, and `buildah` Tasks are installed by the OpenShift Pipelines Operator in the `openshift-pipelines` namespace. This pipeline references them using the cluster resolver, which replaces the deprecated `ClusterTask` resource. You can verify that these Tasks are available by running:

```
oc get task -n openshift-pipelines
```

Lab 3: Test CI Pipeline

In this lab, you'll manually execute the CI pipeline to verify that it successfully clones the application source code, builds the application, retrieves registry credentials from Vault, and pushes the container image to the internal OpenShift registry.

You'll also learn how to monitor pipeline execution and troubleshoot common issues before configuring automated pipeline triggers.

By the end of this lab, you will have:

- Created a PipelineRun to execute the CI pipeline manually
- Monitored the execution of PipelineRuns and TaskRuns
- Verified that the application image was successfully built and pushed
- Learned how to troubleshoot common pipeline failures

Run the CI Pipeline

Before configuring Git webhooks, verify that the pipeline executes successfully by creating a manual PipelineRun.

1. Retrieve the GitLab route by running:

```
GITLAB_URL="https://$(oc get route \
  -n gitlab \
  -l app=webservice \
  -o jsonpath='{.items[0].spec.host}')"

echo "Using GitLab URL: ${GITLAB_URL}"
```

2. Create a PipelineRun:

```
oc create -f - <<EOF
apiVersion: tekton.dev/v1
kind: PipelineRun
metadata:
  generateName: etx-app-base-app-ci-
  namespace: etx-app-dev
spec:
  pipelineRef:
    name: etx-app-base-app-ci
  params:
    - name: git-url
      value: ${GITLAB_URL}/software-factory/etx_app_base_app
    - name: git-revision
      value: main
    - name: image-name
      value: image-registry.openshift-image-registry.svc:5000/etx-app-dev/etx-app-
base-app
  workspaces:
    - name: shared-workspace
      persistentVolumeClaim:
        claimName: pipeline-workspace-pvc
  taskRunTemplate:
    serviceAccountName: pipeline
EOF
```

The PipelineRun starts the CI pipeline using the source code in GitLab and executes each configured task.

Monitor Pipeline Execution

Once the PipelineRun starts, monitor its progress to ensure each task completes successfully.

Watch the PipelineRun Status

The simplest way to monitor execution is to watch the PipelineRun status in real time.

```
oc get pipelinerun -n etx-app-dev -w
```

When the pipeline finishes successfully, its status changes to `Succeeded`.

Sample Output:

```
NAME                        SUCCEEDED   REASON    STARTTIME   COMPLETIONTIME
etx-app-base-app-ci-n252n   Unknown     Running   10s
etx-app-base-app-ci-n252n   Unknown     Running   26s
etx-app-base-app-ci-n252n   Unknown     Running   38s
etx-app-base-app-ci-n252n   Unknown     Running   87s
etx-app-base-app-ci-n252n   True        Succeeded   2m27s       0s
```

```
etx-app-base-app-ci-n252n   True        Succeeded   2m27s       0s
etx-app-base-app-ci-n252n   True        Succeeded   2m28s       1s
```

View TaskRun Details

To inspect the individual tasks executed by the pipeline:

```
# Get the name of the latest PipelineRun
LATEST_PR=$(oc get pipelinerun -n etx-app-dev --sort-by=.metadata.creationTimestamp -o
jsonpath='{.items[-1].metadata.name}')

echo "PipelineRun: $LATEST_PR"

# List all TaskRuns for this PipelineRun
oc get taskrun -n etx-app-dev -l tekton.dev/pipelineRun=$LATEST_PR
```

To view the logs for a specific TaskRun:

```
# View logs of a specific TaskRun (example: fetch-creds)
oc logs -n etx-app-dev -l tekton.dev/taskRun=<taskrun-name> -f
```



Replace `<taskrun-name>` with one of the TaskRun names returned by the previous command.

Verify the Container Image

After the pipeline completes, verify that the container image was pushed successfully.

```
oc get imagestream etx-app-base-app -n etx-app-dev

oc get imagestreamtag etx-app-base-app:main -n etx-app-dev
```

If both resources exist, the image was successfully built and pushed to the internal OpenShift image registry.

Troubleshoot Common Issues (Optional)

If the pipeline does not complete successfully, the following checks can help identify common problems. Skip this section if you have encountered no issues in previous steps.

Vault Authentication Errors

If the `fetch-creds` task fails with an error similar to:

```
parse error: Invalid numeric literal
```

Verify that Vault is running:

```
oc get pods -n vault
```

Also confirm that:

- The Kubernetes authentication method is configured in Vault.
- The `pipeline` ServiceAccount exists in the `etx-app-dev` project.

```
oc get serviceaccount pipeline -n etx-app-dev
```

Git Clone Errors

If the `git-clone` task cannot resolve the GitLab host, verify the GitLab route:

```
oc get route \
  -n gitlab \
  -l app=webservice \
  -o jsonpath='{.items[0].spec.host}{"\n"}'
```

If necessary, recreate the PipelineRun using the correct GitLab URL.

Image Push Verification

If the pipeline reports success but the image cannot be found, verify the ImageStream:

```
oc get imagestream etx-app-base-app -n etx-app-dev

oc get imagestreamtag etx-app-base-app:main -n etx-app-dev
```

A successful pipeline should create both resources in the `etx-app-dev` project.

Lab 4: Automate CI Pipeline

In this lab, you'll configure Tekton Triggers and GitLab webhooks to automatically start the CI pipeline whenever changes are pushed to the application repository.

You'll create the Tekton Trigger resources, expose an EventListener, configure GitLab to send push events, and verify that commits automatically trigger new PipelineRuns.

By the end of this lab, you will have:

- Created a TriggerBinding to map GitLab webhook payloads to pipeline parameters
- Created a TriggerTemplate that generates PipelineRuns

- Deployed an EventListener to receive GitLab webhook events
- Exposed the EventListener using an OpenShift Route
- Configured a GitLab webhook to trigger the pipeline automatically
- Verified end-to-end CI pipeline automation

Create the Tekton Trigger Resources

Tekton Triggers connect external events, such as GitLab push notifications, to Tekton Pipelines.

In this exercise, you'll create the TriggerBinding, TriggerTemplate, and EventListener resources required to automatically start a new PipelineRun whenever code is pushed to GitLab.

Create the TriggerBinding

The TriggerBinding extracts values from the GitLab webhook payload and maps them to parameters used by the pipeline.

1. In the Web Terminal, run:

```
oc apply -f - <<'EOF'
apiVersion: triggers.tekton.dev/v1beta1
kind: TriggerBinding
metadata:
  name: gitlab-push-binding
  namespace: etx-app-dev
spec:
  params:
    - name: git-url
      value: $(body.repository.git_http_url)
    - name: git-revision
      value: $(body.checkout_sha)
EOF
```

The binding maps:

- The Git repository URL
- The commit SHA used as the pipeline revision

Create the TriggerTemplate

The TriggerTemplate defines the PipelineRun that Tekton creates when a webhook event is received.

1. Run the following command:

```
oc apply -f - <<'EOF'
apiVersion: triggers.tekton.dev/v1beta1
kind: TriggerTemplate
```

```
metadata:
  name: gitlab-push-template
  namespace: etx-app-dev
spec:
  params:
    - name: git-url
    - name: git-revision
  resourcetemplates:
    - apiVersion: tekton.dev/v1
      kind: PipelineRun
      metadata:
        generateName: etx-app-base-app-ci-
      spec:
        pipelineRef:
          name: etx-app-base-app-ci
        params:
          - name: git-url
            value: $(tt.params.git-url)
          - name: git-revision
            value: $(tt.params.git-revision)
          - name: image-name
            value: image-registry.openshift-image-registry.svc:5000/etx-app-
dev/etx-app-base-app
        workspaces:
          - name: shared-workspace
            persistentVolumeClaim:
              claimName: pipeline-workspace-pvc
        taskRunTemplate:
          serviceAccountName: pipeline
EOF
```

The template creates a new PipelineRun using the shared workspace and pipeline ServiceAccount created in the previous labs.

Create the EventListener

The EventListener exposes an HTTP endpoint that receives webhook requests from GitLab.

1. Run:

```
oc apply -f - <<'EOF'
apiVersion: triggers.tekton.dev/v1beta1
kind: EventListener
metadata:
  name: gitlab-listener
  namespace: etx-app-dev
spec:
  serviceAccountName: pipeline
  triggers:
    - name: gitlab-push
```

```
      bindings:
        - ref: gitlab-push-binding
      template:
        ref: gitlab-push-template
EOF
```

The EventListener connects the GitLab webhook to the TriggerBinding and TriggerTemplate.

Expose the EventListener

GitLab must be able to securely reach the EventListener.

1. Create an HTTPS route:

```
oc create route edge el-gitlab-listener \
  --service=el-gitlab-listener \
  --port=http-listener \
  -n etx-app-dev
```

2. Retrieve the EventListener URL:

```
echo "https://$(oc get route el-gitlab-listener -n etx-app-dev -o
jsonpath='{.spec.host}')"
```

3. Sample output:

```
https://el-gitlab-listener-etx-app-dev.apps.cluster-
xxxxx.dynamic.redhatworkshops.io
```

Copy this URL. You'll use it when configuring the GitLab webhook.



GitLab requires HTTPS endpoints for webhooks. The OpenShift Route provides TLS termination for the EventListener.

Configure the GitLab Webhook

To allow GitLab to automatically trigger the pipeline, configure a webhook for the application repository.



Creating GitLab webhooks requires **Maintainer** or **Owner** access to the project. Administrative configuration also requires the GitLab `root` account.

Enable Local Network Webhooks

By default, GitLab blocks webhook requests to internal cluster addresses.

1. Retrieve the GitLab administrator information:

```
GITLAB_URL="https://$(oc get route -n gitlab -l app=webservice -o
jsonpath='{.items[0].spec.host}')"
GITLAB_ROOT_PASSWORD=$(oc get secret -n gitlab gitlab-gitlab-initial-root-password
-o jsonpath='{.data.password}' | base64 -d)

echo "GitLab URL: $GITLAB_URL"
echo "Admin Area: $GITLAB_URL/admin"
echo "Network Settings (Direct): $GITLAB_URL/admin/application_settings/network"
echo ""
echo "Root Username: root"
echo "Root Password: $GITLAB_ROOT_PASSWORD"
```

2. Sign in to GitLab using the built-in `root` administrator account. Click **Standard** tab (not ETX Lab SSO)

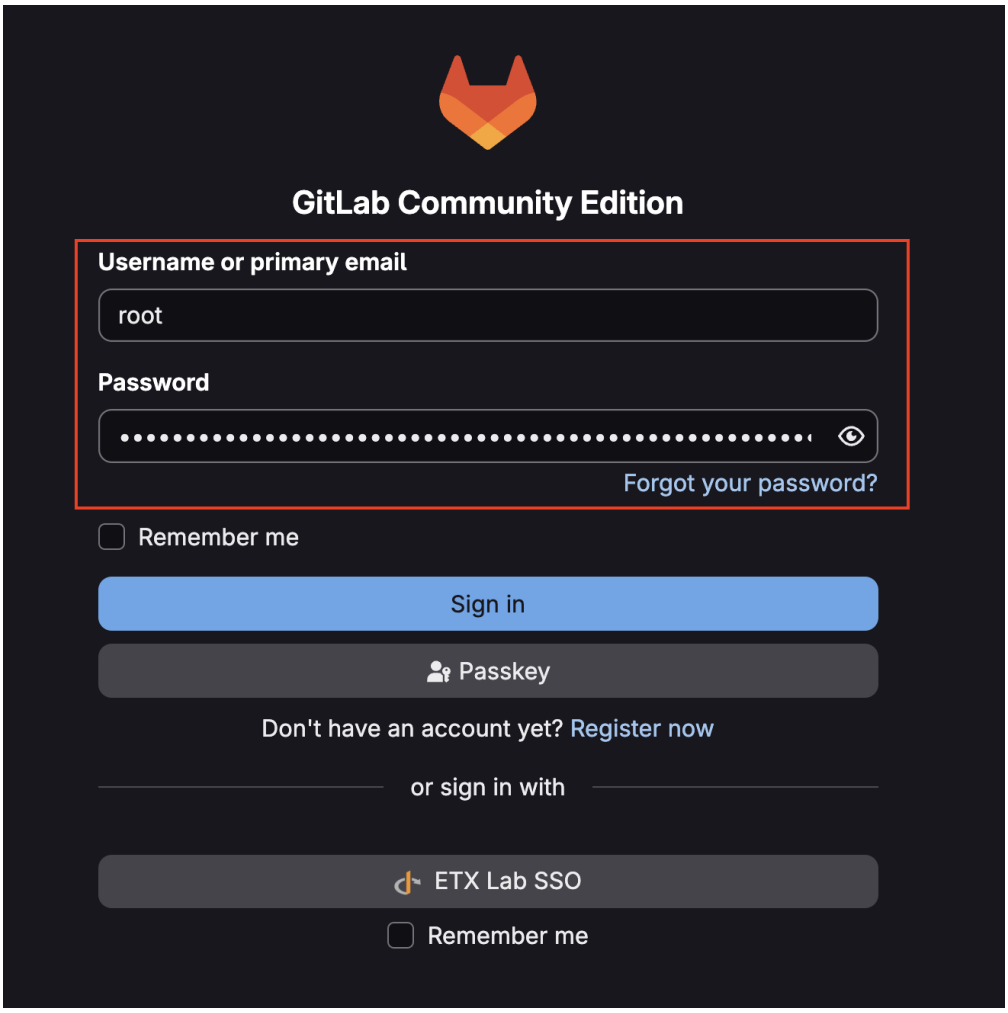


Figure 3. GitLab Login Page(Click the image to view it in full size in a new browser tab)

   - **Username:** `root`
   - **Password:** Use the password from the command above

3. Navigate to:

   - **Admin Area › Settings › Network**

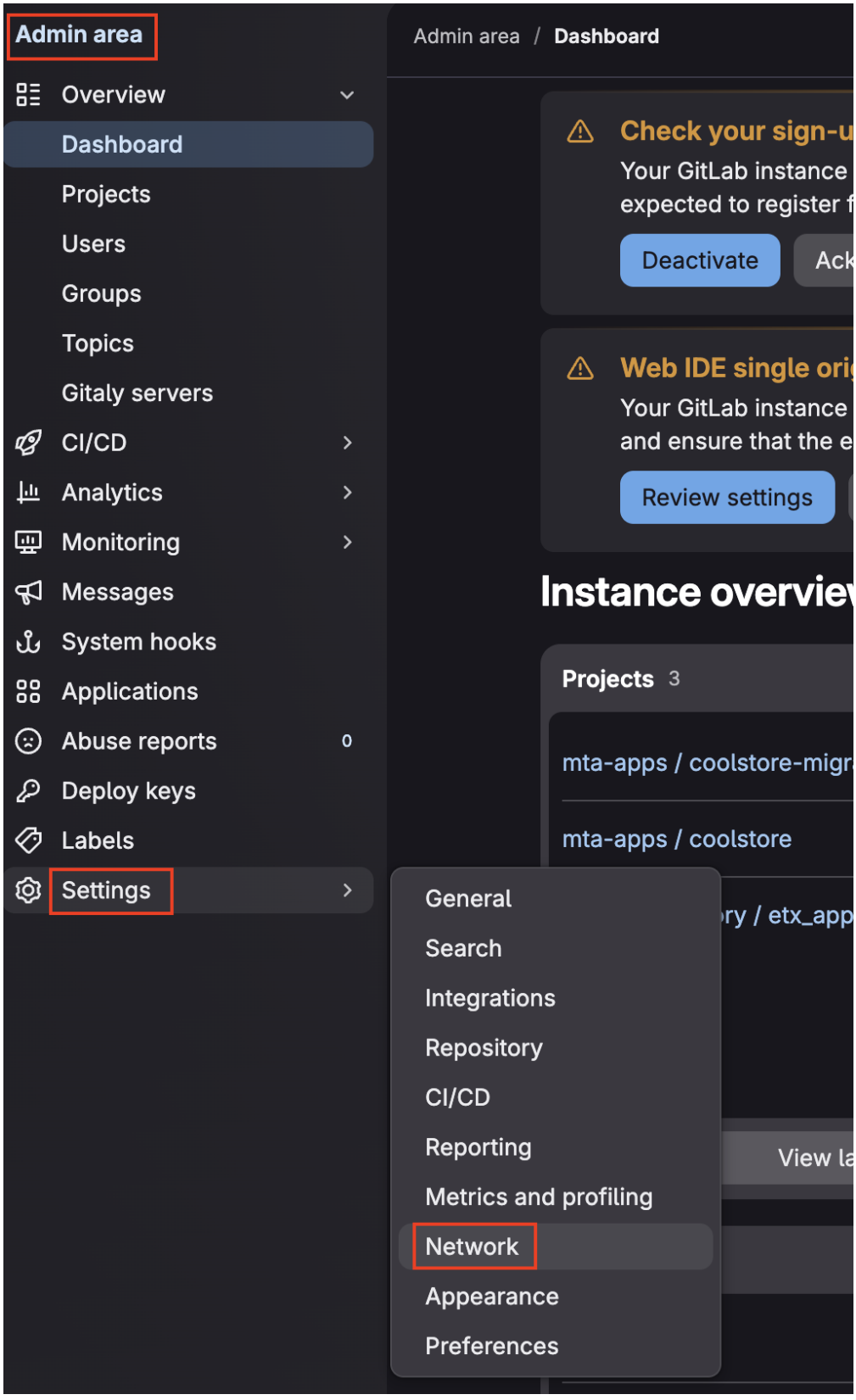


Figure 4. Admin Area Options(Click the image to view it in full size in a new browser tab)

4. Expand **Outbound requests**.

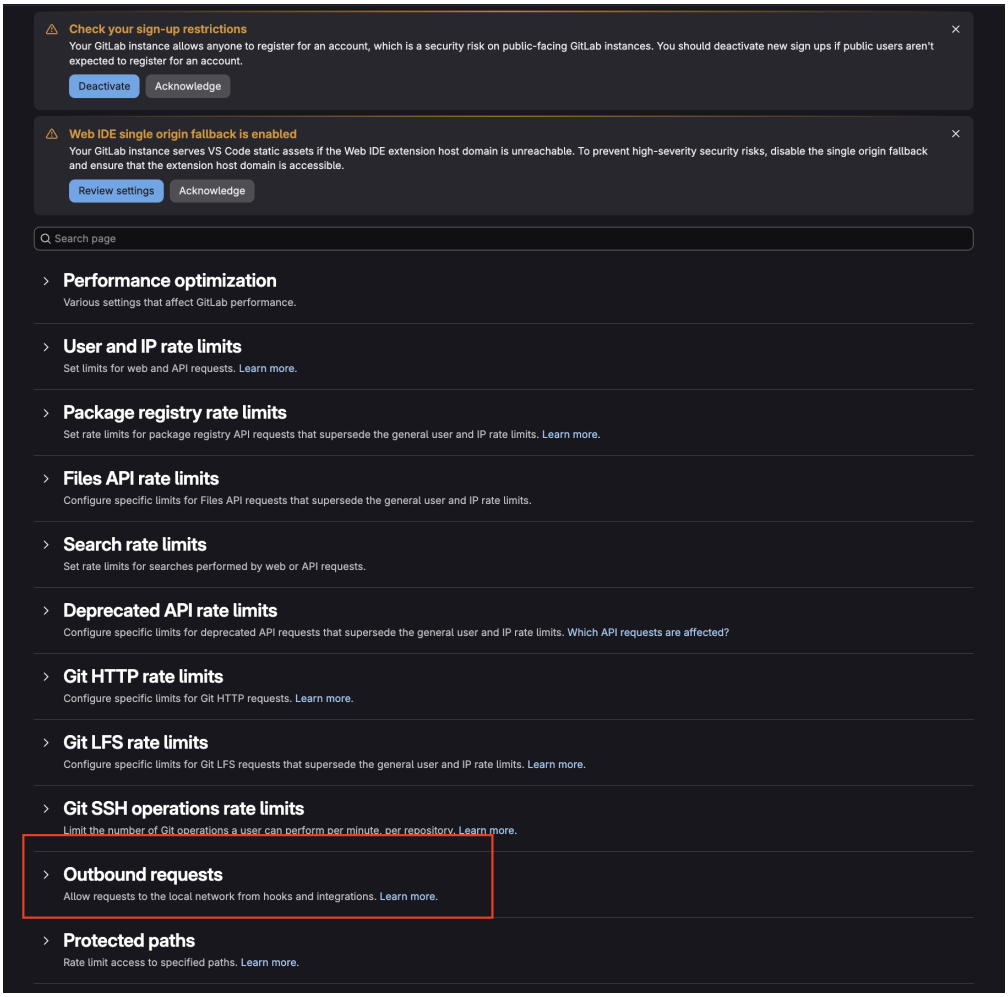


Figure 5. Outbound Requests(Click the image to view it in full size in a new browser tab)

5. Enable local network webhooks:
    - Check ☑ **Allow requests to the local network from webhooks and integrations**
    - Optionally check ☑ **Allow requests to the local network from system hooks**

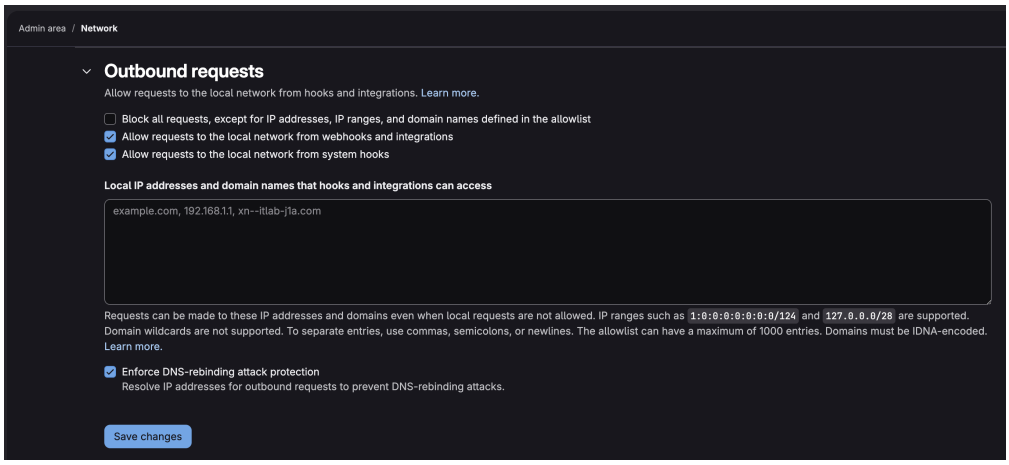


Figure 6. Outbound Settings(Click the image to view it in full size in a new browser tab)

6. Click **[Save]** to save the changes.



This setting allows GitLab to communicate with services running inside the OpenShift cluster, including the Tekton EventListener.

Create the Repository Webhook

1. Navigate back to **[Admin Area]** → **[System hooks]**

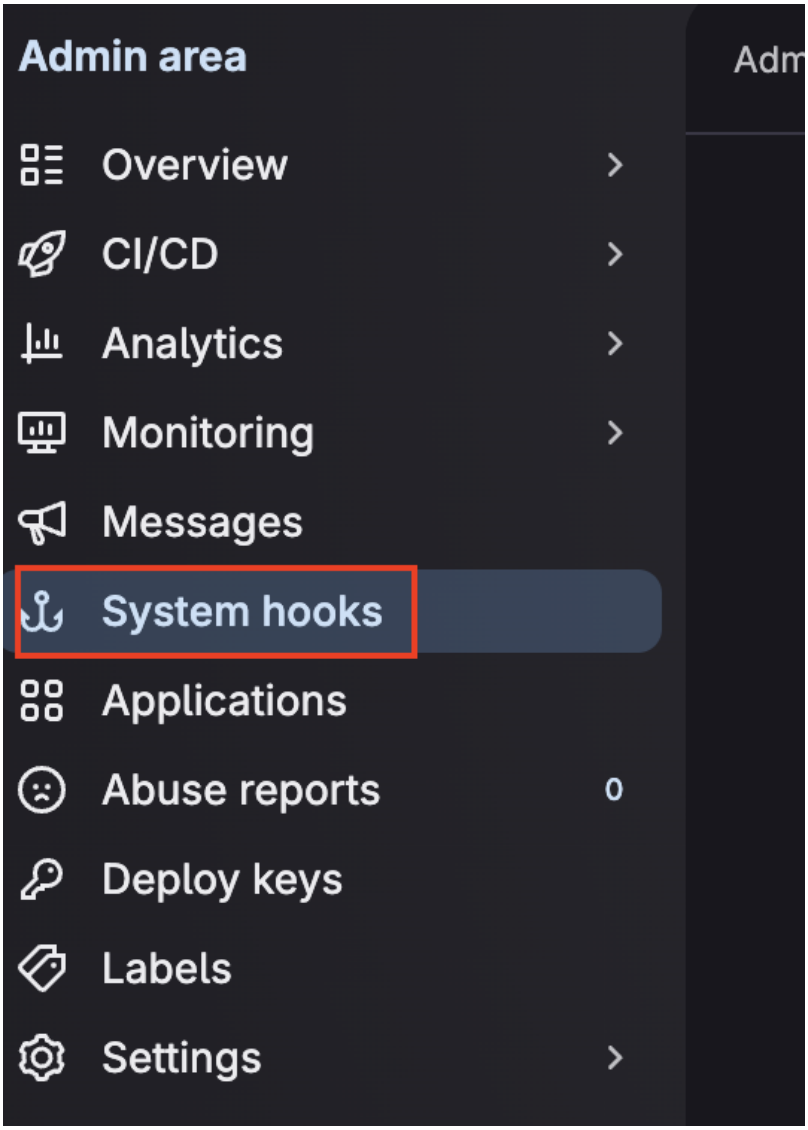


Figure 7. System hooks menu(Click the image to view it in full size in a new browser tab)

2. Click **[Add new webhook]**.

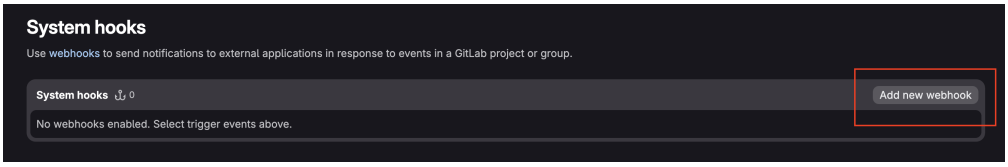


Figure 8. Add new hook(Click the image to view it in full size in a new browser tab)

3. Configure the webhook:
   - **URL**: Paste the EventListener Route URL by retrieving it from below command.

     ```
     echo "https://$(oc get route el-gitlab-listener -n etx-app-dev -o
     jsonpath='{.spec.host}')"
     ```

   - **Trigger**: Check ☑ **Push events**

- **SSL verification**: Uncheck for lab environments with self-signed certificates

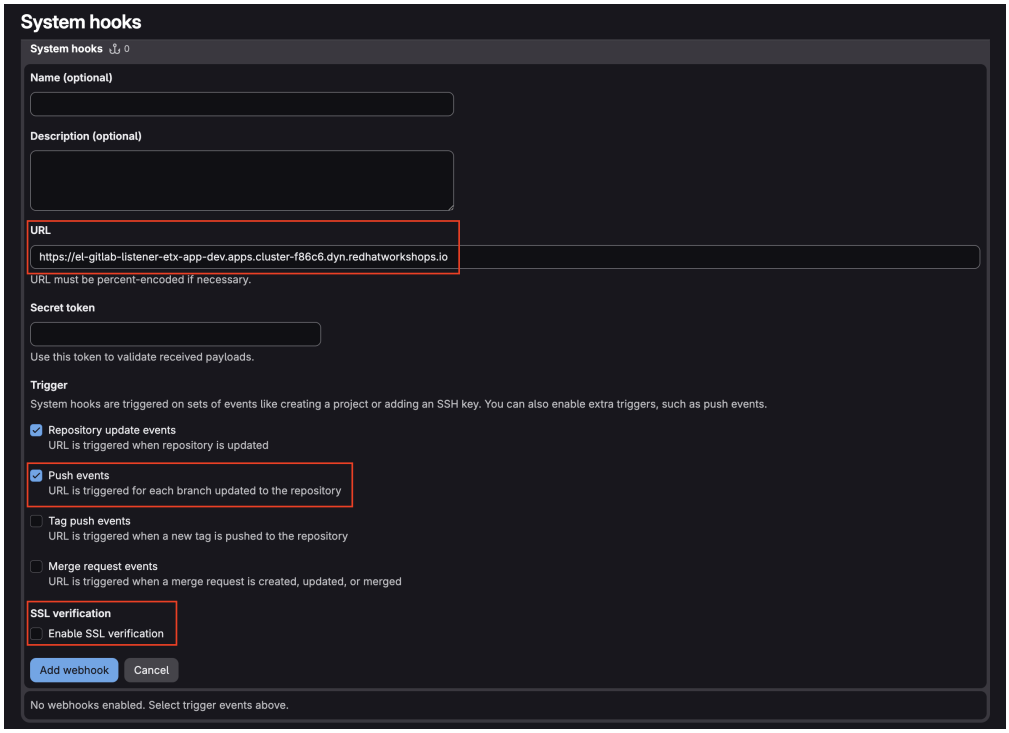


Figure 9. Sample Hook Settings(Click the image to view it in full size in a new browser tab)

4. Click **[Add webhook]**

Test the Webhook

Verify that GitLab can communicate with the EventListener.

1. Select the webhook.
2. Click **[Test]** → **Push events**.

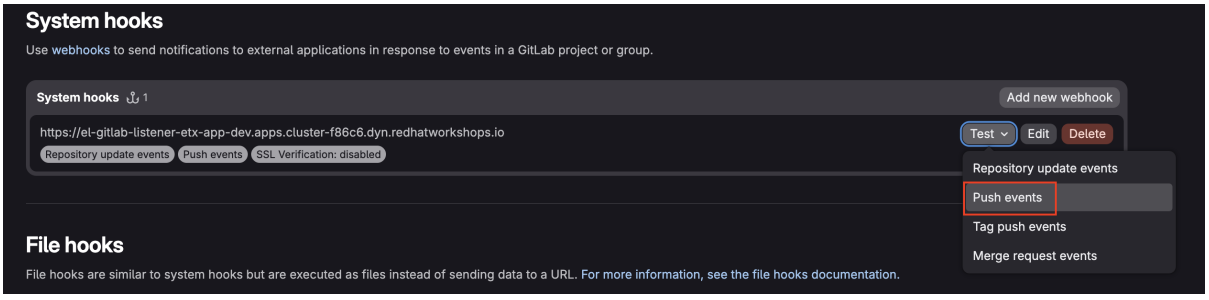


Figure 10. Push events(Click the image to view it in full size in a new browser tab)

3. A successful test returns an HTTP `202 Accepted` response.

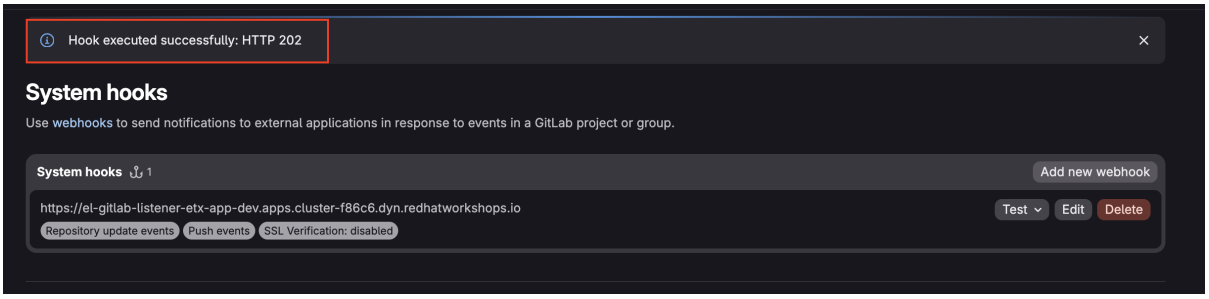


Figure 11. Push test results(Click the image to view it in full size in a new browser tab)

Verify Automatic Pipeline Execution

After the webhook is configured successfully, verify that GitLab automatically starts the CI pipeline.

1. Retrieve the Red Hat Dev Spaces URL by running the following command:

```
oc get checluster devspaces -n openshift-devspaces -o
jsonpath='{.status.cheURL}{"\n"}'
```

2. Open the URL displayed in the output in your web browser and open a terminal inside your workspace.
3. Push a change to your repository and confirm the pipeline triggers:

```
echo "# CI/CD test" >> README.md
git add README.md
git commit -m "Test webhook trigger"
git push origin feature/add-devfile
```

4. Monitor PipelineRuns:

```
oc get pipelinerun -n etx-app-dev -w
```

   A new PipelineRun should be created automatically without manual intervention and should **Succeeded**.

5. Follow the latest run logs:

```
# Get the name of the latest PipelineRun
LATEST_PR=$(oc get pipelinerun -n etx-app-dev --sort-by=.metadata.creationTimestamp
-o jsonpath='{.items[-1].metadata.name}')

echo "Following logs for: $LATEST_PR"

# Follow the logs
oc logs -f $(oc get pods -n etx-app-dev -l tekton.dev/pipelineRun=$LATEST_PR --sort
-by=.metadata.creationTimestamp -o jsonpath='{.items[-1].metadata.name}') -n etx-
```

```
app-dev
```

6. Verify the image was pushed:

```
oc get imagestream etx-app-base-app -n etx-app-dev
```

You should see the image tagged with the commit SHA.

Lab Summary

Throughout these labs, you built a complete continuous integration (CI) workflow using OpenShift Pipelines (Tekton).

Starting with the foundational pipeline resources, you created a reusable CI pipeline that clones application source code, builds and tests the application, securely retrieves registry credentials from HashiCorp Vault, and publishes container images to the OpenShift internal registry. You then automated pipeline execution by integrating GitLab webhooks with Tekton Triggers and verified the complete end-to-end CI workflow.

You now have:

- A Tekton pipeline for building, testing, and containerizing applications
- A shared pipeline workspace and ServiceAccount with the required permissions
- Secure registry credential retrieval using HashiCorp Vault
- Automated pipeline execution through GitLab webhooks and Tekton Triggers
- Container images automatically published to the OpenShift internal registry
- A fully validated, event-driven CI pipeline running on OpenShift

GitOps Continuous Deployment



What's in this section?

In this section, you'll learn how Red Hat OpenShift GitOps and Argo CD provide continuous delivery using a declarative, Git-based approach. You'll configure Argo CD to authenticate using the shared Keycloak OpenID Connect (OIDC) integration, create a GitOps repository to store Kubernetes manifests, and deploy the application using automated synchronization. Finally, you'll connect the Tekton CI pipeline to the GitOps repository so that every successful build automatically updates the application deployment.



You will deploy your **own** Argo CD instance in the `etx-app-dev` namespace. The pre-installed OpenShift GitOps instance in the `openshift-gitops` namespace is used for platform infrastructure and is independent of the application GitOps workflow you'll build in this section.

OpenShift GitOps Overview

In the previous sections, you learned how **OpenShift Pipelines** automates application delivery workflows and how container images are built and published for deployment.

The next step is getting the application into the target environment and ensuring that the environment remains consistent with the desired configuration.

Red Hat OpenShift GitOps provides a declarative approach to continuous delivery using **Argo CD**.

Instead of a CI/CD pipeline directly managing the cluster, GitOps uses Git as the **source of truth** for the desired application state and continuously reconciles the OpenShift cluster with that state.



Think about what should happen after an application is deployed if someone manually changes or deletes one of its resources. Should the platform simply leave the cluster in that state, or should it automatically restore the desired configuration?

The Challenge

A traditional deployment pipeline can apply application manifests to a cluster.

For example, a pipeline might:

- Build an application.
- Create a container image.
- Push the image to a registry.
- Apply Kubernetes manifests.
- Verify that the deployment completed successfully.

However, the pipeline typically finishes after the deployment has been performed. If someone later modifies or deletes a resource, the pipeline does not automatically run again to restore the desired state.

This can result in **configuration drift** between what the application is supposed to look like and what is actually running in the cluster.



A deployment pipeline can change the cluster, but continuous delivery requires more than performing a deployment once. The platform must also continuously ensure that the running environment matches the desired state.

What is OpenShift GitOps?

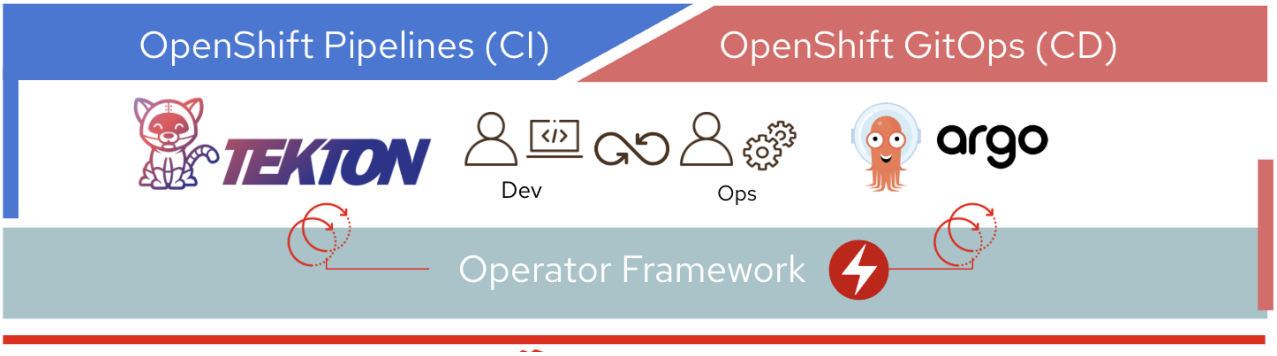


Figure 12. CICD (Click the image to view it in full size in a new browser tab)

OpenShift GitOps provides a **declarative continuous delivery** model based on the open source **Argo CD** project. The desired configuration of an application is stored in Git. OpenShift GitOps continuously compares the configuration stored in Git with the actual state of resources running in the OpenShift cluster. When the two states differ, OpenShift GitOps can synchronize the cluster with the desired state. This process is called **reconciliation**.

Git as the Source of Truth

Application and deployment configuration is stored in Git, providing a version-controlled and auditable definition of the desired state.

Continuous Reconciliation

Argo CD continuously compares the desired state in Git with the live state of the OpenShift

cluster and identifies differences.

Automated Synchronization

When configured for automated synchronization, Argo CD applies the desired configuration to the cluster so that the live environment matches Git.

Declarative Delivery

A key principle of OpenShift GitOps is **declarative configuration**.

Instead of defining a sequence of commands that explains how to deploy an application, GitOps defines the state that the application should have.

For example, a declarative configuration might specify:

```
replicas: 3
image: quay.io/example/app:v2
service:
  port: 8080
```

This configuration describes the **desired state** of the application.

Argo CD continuously works to ensure that the OpenShift cluster reflects that desired state.

The focus changes from:

```
"Run these deployment commands"
```

to:

```
"This is the state the application should have"
```

This makes application delivery more predictable and easier to audit.

Git as the Source of Truth

In a GitOps model, application deployment configuration is stored in a Git repository.

The repository can contain resources such as:

- Kubernetes manifests
- Kustomize configurations
- Helm charts

- Application deployment configuration
- Environment-specific configuration

Git provides version control for the desired application state.

Every change can be reviewed, approved, and tracked through the normal Git workflow.

For example:

```
Git Repository
      |
      | Desired State
      v
   Argo CD
      |
      | Synchronization
      v
OpenShift Cluster
```

This creates a clear relationship between the configuration stored in Git and the resources running in OpenShift.

How Argo CD Works

Argo CD continuously monitors the applications it manages.

It compares two states:

State	Description
Desired state	The application configuration defined in the Git repository.
Live state	The resources currently running in the OpenShift cluster.

When these states match, the application is considered **Synced**.

When they differ, the application is considered **OutOfSync**.

Argo CD can then synchronize the cluster so that the live state matches the desired state.

This continuous comparison is what makes GitOps different from a deployment process that runs only once.

The GitOps Workflow

A typical OpenShift GitOps workflow looks like this:

1. A developer makes a change to the application.
2. The application is built and tested using OpenShift Pipelines.
3. A new container image is created and pushed to an image registry.

4. The desired deployment configuration is updated in Git.
5. Argo CD detects the change in the Git repository.
6. Argo CD compares the desired state with the live cluster state.
7. Argo CD synchronizes the application.
8. OpenShift runs the updated application.

The workflow does not end when the deployment occurs.

Argo CD continues monitoring the application after synchronization.

If the live state later differs from the desired state, Argo CD detects the difference.

Configuration Drift

Configuration drift occurs when the actual state of the environment differs from the desired state.

For example, suppose Git defines:

```
Application replicas = 3
```

An administrator or another process accidentally changes the deployment so that only two replicas are running.

The cluster now differs from the configuration stored in Git.

Argo CD detects this difference and reports the application as **OutOfSync**.

If automated synchronization is enabled, Argo CD can restore the application to the desired configuration.

This provides an important operational capability: the environment can continuously move back toward the state defined in Git.

Self-Healing

GitOps can provide **self-healing** behavior for managed applications.

Consider the following scenario:

1. Git defines an application with three replicas.
2. Argo CD deploys the application with three replicas.
3. Someone accidentally deletes one of the application pods.
4. The Kubernetes controllers recreate the pod according to the deployment configuration.

Now consider a more significant change:

1. Git defines the application configuration.

2. Someone manually modifies a managed resource in the cluster.
3. The live state no longer matches Git.
4. Argo CD detects the difference.
5. Argo CD synchronizes the application according to the configured synchronization policy.

This allows GitOps to continuously protect the desired application configuration rather than relying on a deployment pipeline to run again.



GitOps does not simply deploy an application. It continuously observes and reconciles the application against the desired state defined in Git.

OpenShift Pipelines and OpenShift GitOps

OpenShift Pipelines and OpenShift GitOps have complementary responsibilities.

OpenShift Pipelines focuses on **building and delivering application changes**.

OpenShift GitOps focuses on **maintaining the desired application state**.

A typical workflow looks like this:

```
Developer
    |
    v
Application Source
    |
    v
OpenShift Pipelines
    |
    +--> Build
    |
    +--> Test
    |
    +--> Build Image
    |
    v
Image Registry
    |
    v
Update Desired State
    |
    v
Git Repository
    |
    v
OpenShift GitOps
    |
    v
```

```
OpenShift Cluster
```

The important distinction is that the pipeline does not need to continuously manage the cluster after updating the desired state.

Instead, the pipeline can update the Git repository and allow OpenShift GitOps to take responsibility for synchronizing the environment.



OpenShift Pipelines automates the **delivery workflow**, while OpenShift GitOps continuously manages the **desired state** of the application in the cluster.

Why GitOps?

OpenShift GitOps provides several benefits for application delivery and operations.

For developers:

- Use Git-based workflows for application deployment configuration.
- Track deployment changes through version control.
- Reduce manual deployment steps.
- Promote application changes through controlled configuration changes.

For Platform Engineers and Operations teams:

- Manage applications declaratively.
- Continuously detect configuration drift.
- Automate application synchronization.
- Improve consistency across environments.
- Maintain an auditable history of configuration changes.
- Reduce manual cluster changes.

For organizations:

- Establish Git as a source of truth.
- Improve deployment consistency.
- Increase traceability and auditability.
- Reduce configuration drift.
- Enable scalable continuous delivery practices.

OpenShift GitOps in the Software Factory

Within a **Software Factory**, OpenShift GitOps provides the continuous delivery capability that connects application configuration in Git with running applications on OpenShift.

A complete Software Factory workflow can therefore look like this:

1. Developers develop applications in a standardized development environment.
2. Developers commit source code to Git.
3. OpenShift Pipelines builds and tests the application.
4. A container image is created and published.
5. The desired deployment configuration is updated in Git.
6. OpenShift GitOps detects the desired-state change.
7. Argo CD synchronizes the application to OpenShift.
8. Argo CD continues monitoring the application.
9. Any configuration drift can be detected and reconciled.

This creates an automated path from **application source code to a continuously managed application environment**.

Everything as Code

OpenShift GitOps supports the broader Software Factory principle of **everything-as-code**.

Application source code, deployment configuration, environment configuration, and delivery definitions can all be managed through version-controlled repositories.

This provides:

- **Version control** — Changes are tracked over time.
- **Auditability** — Teams can identify who changed the desired state and when.
- **Repeatability** — The same configuration can be applied consistently.
- **Reviewability** — Changes can go through pull requests and approval workflows.
- **Recovery** — The desired configuration remains available in Git.



GitOps changes deployment from a one-time action into a continuous process where the desired state is stored in Git and the running environment is continuously reconciled with it.

Benefits

For developers:

- Deploy applications using familiar Git workflows.
- Reduce manual deployment activities.
- Track application configuration changes.
- Use consistent deployment processes.

For Platform Engineers:

- Manage application environments declaratively.

- Standardize deployment configuration.
- Detect and manage configuration drift.
- Provide automated synchronization.
- Improve operational consistency.

For organizations:

- Establish Git as the source of truth.
- Improve deployment reliability.
- Increase visibility and auditability.
- Reduce manual cluster changes.
- Support repeatable application delivery across environments.

Reflection

▼ **Think about your own organization (Click to expand)**

[ROOT\:mini illustration lightbulb red RH.Small\ square\ transparent] | *ROOT\:mini-illustration-lightbulb-red-RH.Small_square_transparent.png*

[ROOT\:mini illustration lightbulb red RH.Small\ square\ transparent] | *ROOT\:mini-illustration-lightbulb-red-RH.Small_square_transparent.png*

- How are applications currently deployed to OpenShift?
- What happens when someone manually changes a deployed resource?
- Where is the desired configuration of your applications stored?
- How could Git become the source of truth for application deployments?
- How could continuous reconciliation reduce configuration drift?

```
<div class="key-card">
```

Key Takeaway

OpenShift GitOps provides continuous delivery using *Argo CD and a declarative, Git-based approach. Git serves as the source of truth for the desired application state, while Argo CD continuously compares and reconciles that state with the OpenShift cluster. Together with OpenShift Pipelines, it creates a repeatable path from application source code to a continuously managed application environment.

```
</div>
```

Lab 5: GitOps Authentication

In this lab, you'll verify that the OpenShift GitOps Operator is installed and configure OpenID Connect (OIDC) authentication for Argo CD using the shared Keycloak client.

By the end of this lab, you will have:

- Verified that the OpenShift GitOps Operator is installed
- Configured Argo CD to use the shared Keycloak OIDC client
- Created the OIDC secret required for Argo CD authentication

Verify the OpenShift GitOps Operator

The OpenShift GitOps Operator is pre-installed as part of the lab environment. Before creating GitOps resources, verify that the operator is available and running successfully.

1. Verify that the OpenShift GitOps Operator is installed:

```
oc get csv -n openshift-operators | grep gitops
```

Confirm that the `openshift-gitops-operator` ClusterServiceVersion (CSV) is listed with the `Succeeded` phase.



If the operator is not yet in the `Succeeded` phase, wait a few moments and rerun the command before proceeding.

Configure Keycloak Authentication for Argo CD

The ETX platform uses a shared Keycloak OpenID Connect (OIDC) client named `argocd` for all Argo CD instances. This enables users to authenticate to Argo CD using the same enterprise credentials they use for GitLab, Vault, and other platform services.

In this exercise, you'll create the Kubernetes Secret that stores the shared OIDC client credentials.

1. Run the following command to create the Argo CD OIDC secret:

```
# Get Keycloak URL (etx-sso route)
KEYCLOAK_URL="https://etx-sso.apps.$(oc get route -n etx-keycloak -o
jsonpath='{.items[0].spec.host}' | sed 's/etx-sso\.apps\.//')"

# Use the shared argocd client secret from etx-gitops
CLIENT_SECRET="changeme"

oc create secret generic argocd-oidc-secret -n etx-app-dev \
  --from-literal=clientSecret=${CLIENT_SECRET}

# Add labels to help ArgoCD operator recognize the secret
oc label secret argocd-oidc-secret -n etx-app-dev \
  app.kubernetes.io/name=argocd-oidc-secret \
  app.kubernetes.io/part-of=argocd

echo "Keycloak issuer: ${KEYCLOAK_URL}/realms/etx-lab"
```



Use the value `changeme` exactly as shown. In this lab environment, it is the shared client secret configured for the existing `argocd` OIDC client. Do **not** replace it with another value.

The command creates the `argocd-oidc-secret` Secret in the `etx-app-dev` namespace and labels it so it can be used by the OpenShift GitOps Operator.



The `argocd` OIDC client is shared across the ETX platform. Do **not** create a new client in Keycloak. Use the existing client and the shared client secret provided for this lab.

Lab 6: Deploy Argo CD

In this lab, you'll deploy an Argo CD instance in your development project and configure it to use the shared Keycloak OpenID Connect (OIDC) integration created in the previous lab.

By the end of this lab, you will have:

- Deployed an Argo CD instance in the `etx-app-dev` project
- Configured Argo CD to authenticate using the shared Keycloak OIDC integration
- Verified that the Argo CD components and Route are available
- Accessed the Argo CD dashboard using Keycloak Single Sign-On

Deploy the Argo CD Instance

In this exercise, you'll deploy an Argo CD instance that will be used to manage application deployments throughout the remaining labs.

1. Retrieve the cluster domain:

```
CLUSTER_DOMAIN="$(oc get route -n etx-keycloak -o jsonpath='{.items[0].spec.host}'
| sed 's/etx-sso\.apps\.//')"
echo "Cluster domain: ${CLUSTER_DOMAIN}"
```

2. Deploy the Argo CD instance:

```
CLUSTER_DOMAIN="$(oc get route -n etx-keycloak -o jsonpath='{.items[0].spec.host}'
| sed 's/etx-sso\.apps\.//')"

cat <<EOF | oc apply -f -
apiVersion: argoproj.io/v1beta1
kind: ArgoCD
metadata:
  name: argocd
  namespace: etx-app-dev
spec:
```

```
  server:
    autoscale:
      enabled: false
    grpc:
      ingress:
        enabled: false
    ingress:
      enabled: false
    insecure: true
    resources:
      limits:
        cpu: 500m
        memory: 256Mi
      requests:
        cpu: 125m
        memory: 128Mi
    route:
      enabled: true
      tls:
        termination: edge
        insecureEdgeTerminationPolicy: Redirect
    service:
      type: ClusterIP
  grafana:
    enabled: false
    ingress:
      enabled: false
    route:
      enabled: false
  prometheus:
    enabled: false
    ingress:
      enabled: false
    route:
      enabled: false
  initialSSHKnownHosts: {}
  rbac:
    defaultPolicy: 'role:readonly'
    policy: |
      g, platform-admin, role:admin
      g, platform-developer, role:admin
    scopes: '[groups, preferred_username]'
  extraConfig:
    oidc.config: |
      name: ETX Keycloak
      issuer: https://etx-sso.apps.${CLUSTER_DOMAIN}/realms/etx-lab
      clientID: argocd
      clientSecret: \$argocd-oidc-secret:clientSecret
      requestedScopes:
        - openid
      requestedIDTokenClaims:
```

```
        groups:
          essential: true
      logoutURL: https://etx-sso.apps.${CLUSTER_DOMAIN}/realms/etx-
lab/protocol/openid-connect/logout
  repo:
    resources:
      limits:
        cpu: 1000m
        memory: 1Gi
      requests:
        cpu: 250m
        memory: 256Mi
  resourceExclusions: |
    - apiGroups:
      - tekton.dev
      clusters:
      - '*'
      kinds:
      - TaskRun
      - PipelineRun
  controller:
    processors: {}
    resources:
      limits:
        cpu: 2000m
        memory: 2Gi
      requests:
        cpu: 250m
        memory: 1Gi
  ha:
    enabled: false
    resources:
      limits:
        cpu: 500m
        memory: 256Mi
      requests:
        cpu: 250m
        memory: 128Mi
EOF
```

The command deploys an Argo CD instance named `argocd` in the `etx-app-dev` project.



The `server.insecure: true` setting configures Argo CD to serve HTTP traffic internally while the OpenShift Route performs TLS termination. This prevents redirect loops (`ERR_TOO_MANY_REDIRECTS`) when using edge TLS termination.



The Argo CD custom resource references the `argocd-oidc-secret` created in the previous lab, enabling authentication through the shared Keycloak OIDC client.

Verify the Argo CD Deployment

Wait for the Argo CD components to start.

1. Run the following command:

```
oc get pods -n etx-app-dev -w
```

Wait until all `argocd-*` pods report a status of `1/1 Ready`, then press `Ctrl` + `C` to stop watching.

Once all pods are running, your Argo CD instance is ready to manage GitOps applications in the following labs.

Access the Argo CD Dashboard

Retrieve the Argo CD dashboard URL and sign in to verify that your deployment is working correctly.

1. Get the Argo CD URL:

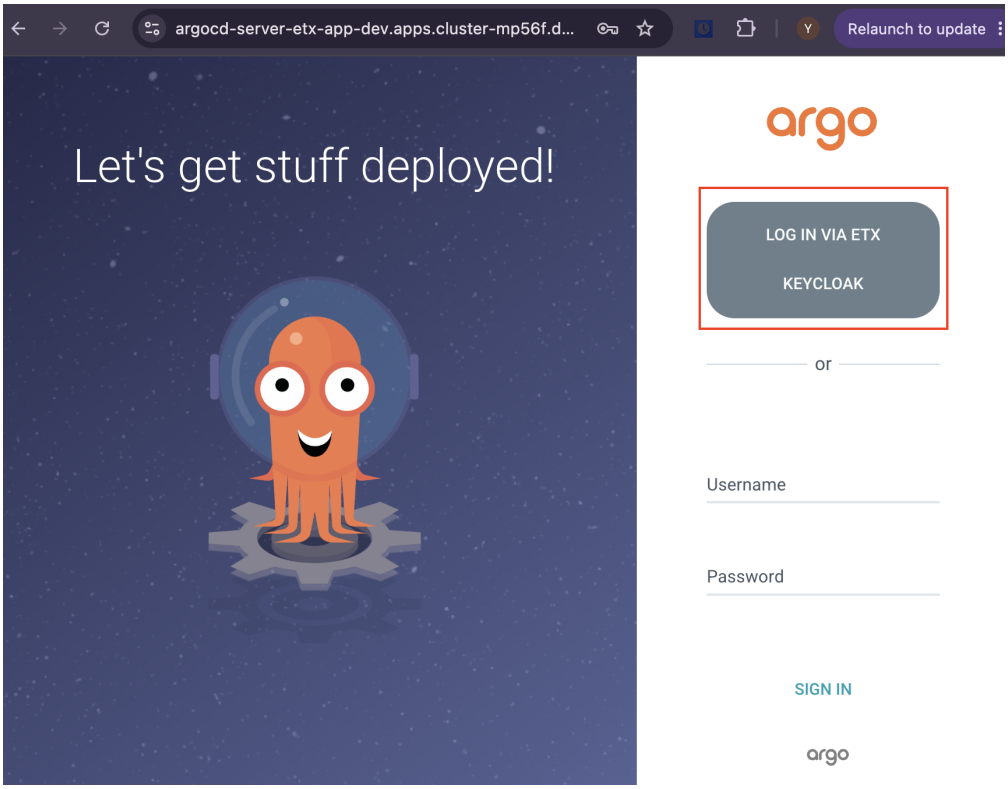
```
echo "https://$(oc get route argocd-server -n etx-app-dev -o
jsonpath='{.spec.host}')"
```

2. Open the displayed URL in your browser.

You can authenticate using either **ETX Keycloak Single Sign-On** or the built-in Argo CD administrator account.

Sign in with ETX Keycloak (Recommended)

1. On the Argo CD login page, select **ETX Keycloak**.



2. Sign in using one of the lab credentials:
   - `platform-admin` / `openshift`
   - `platform-developer` / `openshift`
3. After successful authentication, you'll be redirected to the Argo CD dashboard.



Keycloak Single Sign-On provides the same authentication experience used throughout the ETX platform, including GitLab, Quay, and Vault.

Sign in with the Admin Account (Optional)

If you prefer to use the built-in administrator account:

1. Retrieve the default admin password:

```
oc get secret argocd-cluster -n etx-app-dev \
  -o jsonpath='{.data.admin\.password}' | base64 -d && echo
```

2. Sign in using:
   - **Username:** `admin`
   - **Password:** Output from the previous command

Troubleshooting Dashboard Access

If you receive an `ERR_TOO_MANY_REDIRECTS` error when opening the Argo CD dashboard, verify that the Argo CD server is configured to run in insecure mode behind the OpenShift TLS-terminating route.

1. Confirm the current setting:

```
oc get argocd argocd -n etx-app-dev \
  -o jsonpath='{.spec.server.insecure}{"\n"}'
```

   The command should return:

```
true
```

2. If the value is not `true`, update the Argo CD instance:

```
oc patch argocd argocd -n etx-app-dev --type=merge \
  -p '{"spec":{"server":{"insecure":true}}}'
```

3. Wait for the Argo CD server pod to restart:

```
oc get pods -n etx-app-dev \
  -l app.kubernetes.io/name=argocd-server -w
```

   Press `Ctrl` + `C` once the new pod reports `1/1 Ready`.



If the redirect error persists after the pod restarts, clear your browser cache or open the dashboard in an incognito/private browsing window before trying again.

Lab 7: GitOps Repository

In this lab, you'll create a GitOps repository in GitLab to store the Kubernetes manifests that define your application deployment. This repository will serve as the source of truth that Argo CD continuously monitors and synchronizes with your OpenShift cluster.

By the end of this lab, you will have:

- Created a GitOps repository in GitLab
- Organized Kubernetes manifests for the development environment
- Committed and pushed the initial application manifests

Create the GitOps Repository

In this exercise, you'll create a new GitLab repository that will store the Kubernetes manifests used by Argo CD for GitOps-based application deployment.

1. Retrieve the GitLab URL:

```
# Get GitLab URL
```

```
GITLAB_URL="https://$(oc get route -n gitlab \
  -l app=webservice \
  -o jsonpath='{.items[0].spec.host}')"

echo "GitLab URL: ${GITLAB_URL}"
```

2. Open the displayed GitLab URL in your browser.
3. Click **[Project]** → **[New Project]**.

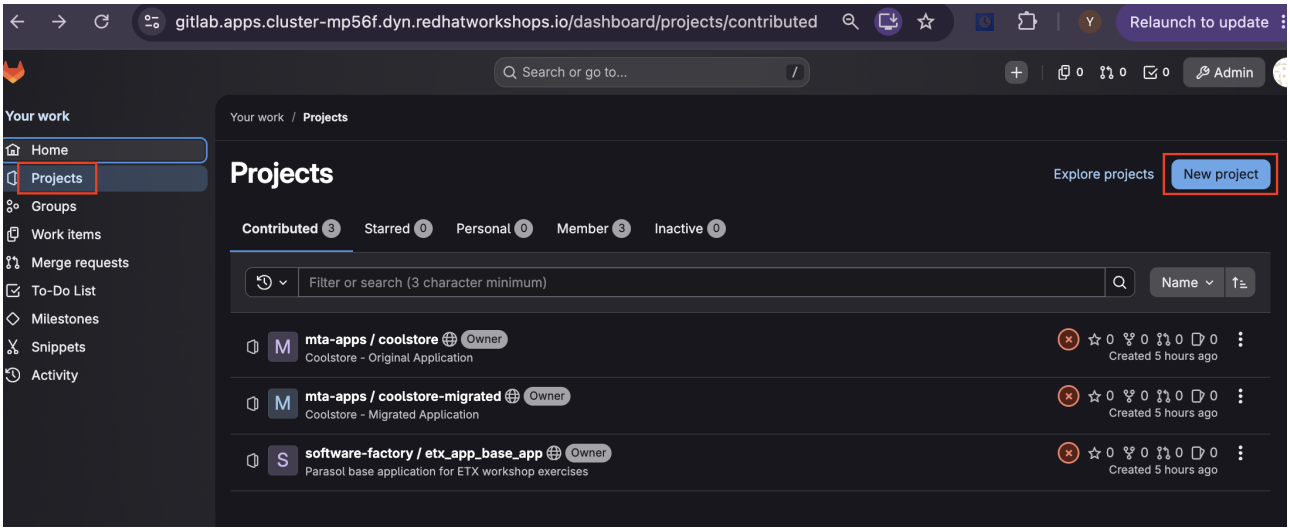


Figure 13. Create new project(Click the image to view it in full size in a new browser tab)

4. Select **[Create blank project]**

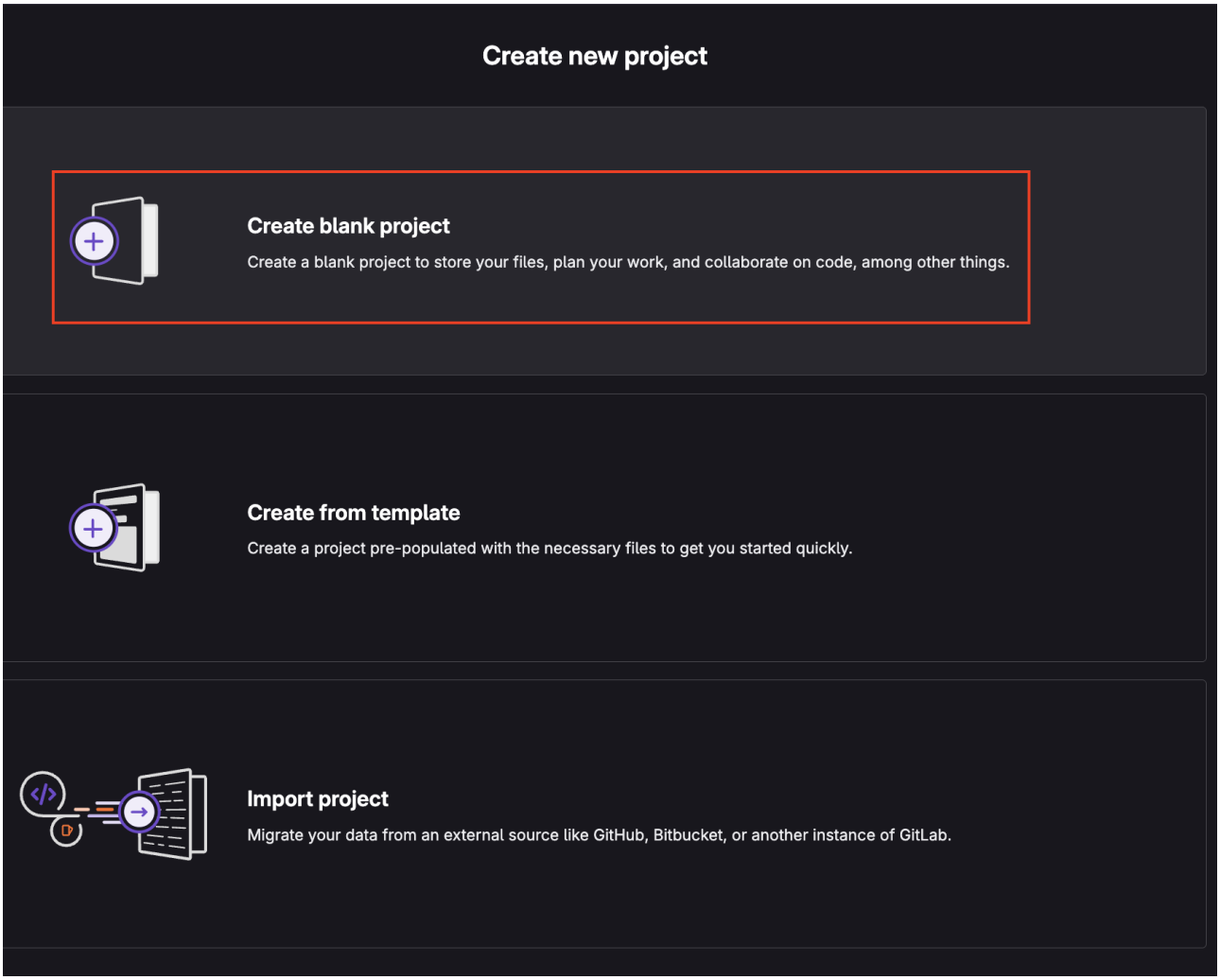


Figure 14. Create blank project(Click the image to view it in full size in a new browser tab)

5. Create a new project named `etx-app-gitops` in the `software-factory` group.

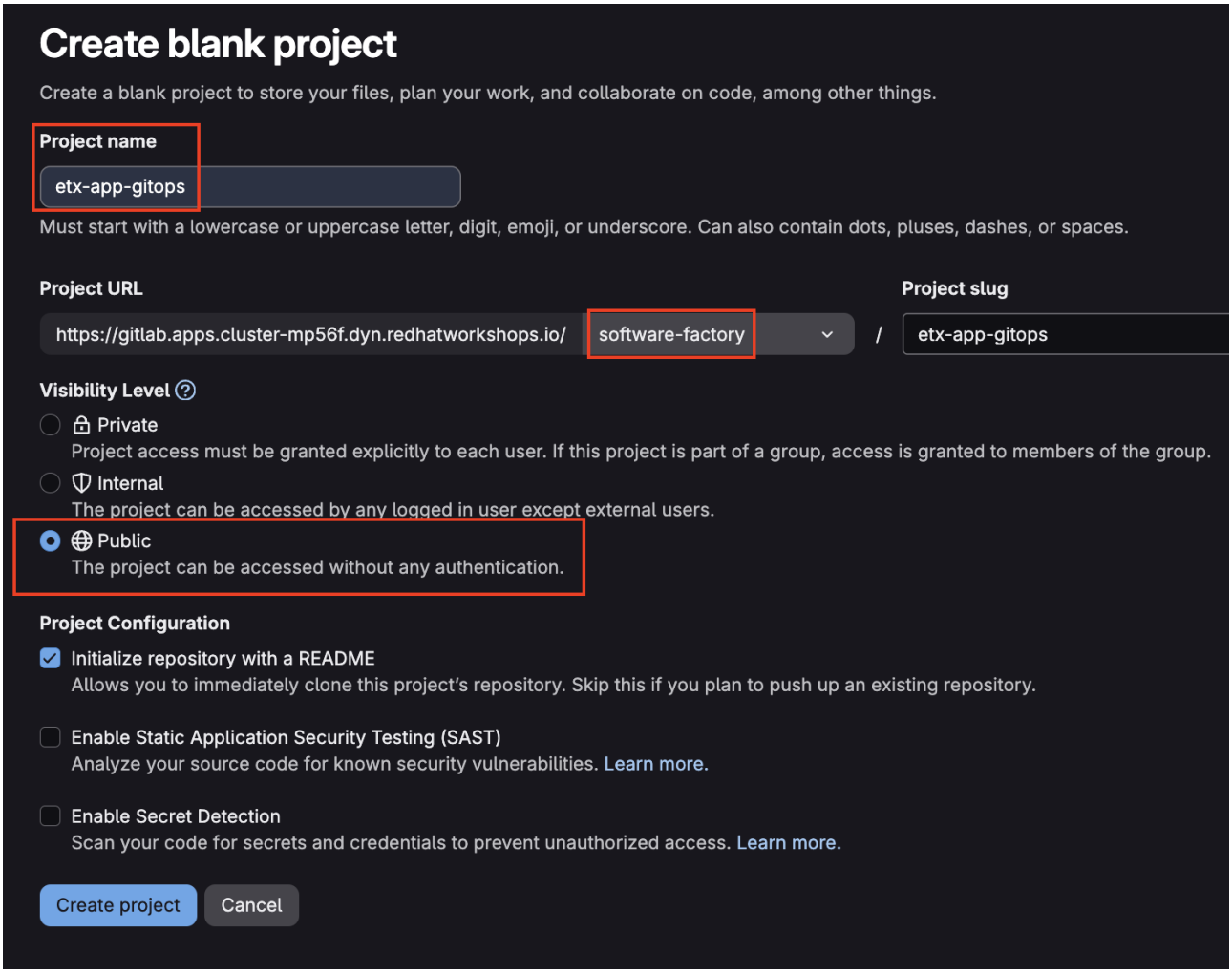


Figure 15. Project details(Click the image to view it in full size in a new browser tab)

6. Clone the repository locally:

```
git clone ${GITLAB_URL}/software-factory/etx-app-gitops.git
cd etx-app-gitops
```

The repository will use the following directory structure:

```
etx-app-gitops/
├── environments/
│   ├── dev/
│   │   ├── namespace.yaml
│   │   ├── deployment.yaml
│   │   ├── service.yaml
│   │   └── route.yaml
│   └── stage/
│       ├── namespace.yaml
│       ├── deployment.yaml
│       ├── service.yaml
│       └── route.yaml
└── README.md
```

Create the Development Environment Manifests

Create the Kubernetes manifests that define the application's development environment.

1. Confirm you are inside the **etx-app-gitops** directory that you cloned in previous step.
2. Once you are inside the cloned repository, create **environments/dev/deployment.yaml** as shown below:

```
mkdir -p environments/dev
cat > environments/dev/deployment.yaml <<'EOF'
apiVersion: apps/v1
kind: Deployment
metadata:
  name: etx-app-base-app
  namespace: etx-app-dev
  labels:
    app: etx-app-base-app
  annotations:
    instrumentation.opentelemetry.io/inject-java: "true" # (1)
spec:
  replicas: 1
  selector:
    matchLabels:
      app: etx-app-base-app
  template:
    metadata:
      labels:
        app: etx-app-base-app
      annotations:
        instrumentation.opentelemetry.io/inject-java: "true" # (1)
    spec:
      containers:
        - name: etx-app-base-app
          image: image-registry.openshift-image-registry.svc:5000/etx-app-dev/etx-
app-base-app:main # (2)
          ports:
            - containerPort: 8080
          resources:
            requests:
              cpu: 250m
              memory: 512Mi
            limits:
              cpu: "1"
              memory: 1Gi
          env:
            - name: POSTGRESQL_HOST
              value: "parasol-db.etx-app-dev.svc.cluster.local"
            - name: POSTGRESQL_DATABASE
              value: "parasol"
            - name: POSTGRESQL_USER
```

```
              value: "parasol"
            - name: POSTGRESQL_PASSWORD
              value: "parasol"
            - name: KAFKA_BOOTSTRAP_SERVERS
              value: "parasol-kafka-kafka-bootstrap.etx-app-
dev.svc.cluster.local:9092"
EOF
```



The OpenTelemetry annotation automatically injects the Java agent using the Instrumentation custom resource created during the supporting services setup.



The container image tag is updated automatically by the CI pipeline after a successful build.

1. Create the Service manifest named **environments/dev/service.yaml**:

```
cat > environments/dev/service.yaml <<'EOF'
apiVersion: v1
kind: Service
metadata:
  name: etx-app-base-app
  namespace: etx-app-dev
spec:
  selector:
    app: etx-app-base-app
  ports:
    - port: 8080
      targetPort: 8080
EOF
```

2. Create the Route manifest named **environments/dev/route.yaml**:

```
cat > environments/dev/route.yaml <<'EOF'
apiVersion: route.openshift.io/v1
kind: Route
metadata:
  name: etx-app-base-app
  namespace: etx-app-dev
spec:
  to:
    kind: Service
    name: etx-app-base-app
  port:
    targetPort: 8080
  tls:
    termination: edge
EOF
```

Commit the Initial Manifests

Commit the development environment manifests and push them to the GitOps repository.

1. Configure your Git identity:

```
git config --global user.email "platform-developer@example.com"
git config --global user.name "Platform Developer"
```

2. Retrieve the GitLab administrator credentials:

```
GITLAB_ROOT_PASSWORD=$(oc get secret \
  -n gitlab \
  gitlab-gitlab-initial-root-password \
  -o jsonpath='{.data.password}' | base64 -d)

echo "Root Username: root"
echo "Root Password: ${GITLAB_ROOT_PASSWORD}"
```

3. Create a GitLab Personal Access Token for Git authentication:

   GitLab requires a Personal Access Token instead of password for Git operations.

   a. Open the GitLab URL in your browser and sign in with:
      - Username: `root`
      - Password: Use the password from the previous step
   b. Click your avatar (top-right) → **[Edit profile]** (or **[Preferences]**)
   c. In the left sidebar, click **[Access]** → **[Personal access tokens]** → **[Add new tokens]**
   d. Fill in the token details:
      - **Token name**: `git-push`
      - **Expiration date**: Select a future date (e.g., one month from today)
      - **Select scopes**: Check the following boxes:
        - ☑ **read_repository** - Grants read-only access to repositories
        - ☑ **write_repository** - Grants read-write access to repositories

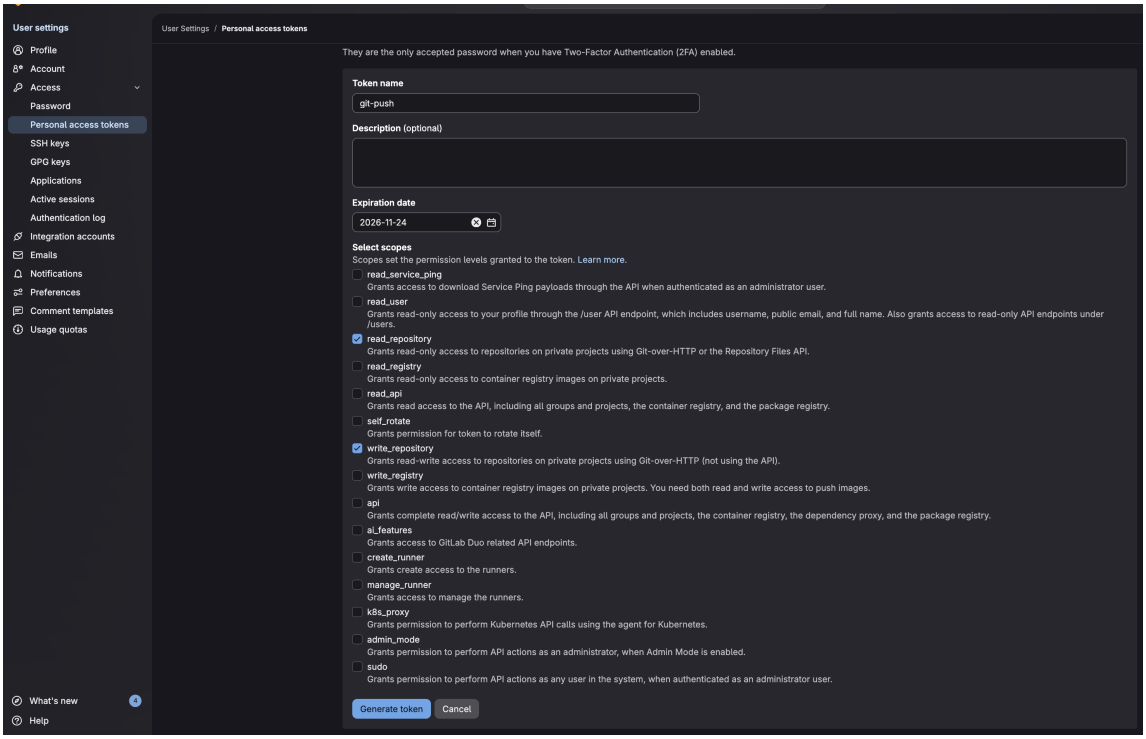


Figure 16. GitLab Personal Access Token creation (Click the image to view it in full size in a new browser tab)

e. Click **[Create personal access token]**

f. **IMPORTANT**: Copy the generated token immediately (it starts with `glpat-`). You will not be able to see it again.



Save the token in a safe location. You will use it as your password when pushing to GitLab.

4. Update the Git remote URL to include your Personal Access Token:

```
# Replace <YOUR-TOKEN> with the token you just created
git remote set-url origin https://root:<YOUR-TOKEN>@$(oc get route -n gitlab -l
app=webservice -o jsonpath='{.items[0].spec.host}')/software-factory/etx-app-
gitops.git
```

Example:

```
# Example with token (yours will be different)
git remote set-url origin https://root:glpat-
Jw08Xm5NouHHEEgIVgvXS286MQp1OjEH@gitlab.apps.cluster-
lhpfm.dyn.redhatworkshops.io/software-factory/etx-app-gitops.git
```

5. Commit and push the manifests:

```
git add -A
git commit -m "feat: add dev environment manifests"
```

```
git push -u origin main
```

The push should now succeed without prompting for credentials since the token is embedded in the remote URL.

Once the changes are pushed, the GitOps repository contains the Kubernetes manifests that Argo CD will use as the desired state for the development environment in the following labs.

Lab 8: Deploy with Argo CD

In this lab, you'll create an Argo CD Application that continuously synchronizes the GitOps repository with your OpenShift cluster. Once configured, Argo CD automatically deploys the application and keeps the cluster aligned with the desired state stored in Git.

By the end of this lab, you will have:

- Granted Argo CD permission to manage resources in the development project
- Created an Argo CD Application for the development environment
- Verified that the application was synchronized and deployed successfully

Grant Argo CD Permissions

Before Argo CD can deploy applications, its application controller requires permission to manage resources in the `etx-app-dev` project.

Grant the required permissions:

```
oc adm policy add-role-to-user admin \
  system:serviceaccount:etx-app-dev:argocd-application-controller \
  -n etx-app-dev
```

This grants the Argo CD Application Controller administrative access within the `etx-app-dev` project so it can create, update, and remove application resources during synchronization.

Create the Argo CD Application

Next, create an Argo CD Application that monitors the GitOps repository and synchronizes the manifests for the development environment.

Create the Argo CD Application:

```
oc apply -f - <<EOF
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: etx-app-dev
  namespace: etx-app-dev
```

```
  labels:
    app.kubernetes.io/part-of: etx_app_base_app
spec:
  project: default
  source:
    repoURL: ${GITLAB_URL}/software-factory/etx-app-gitops.git
    targetRevision: main
    path: environments/dev
  destination:
    server: https://kubernetes.default.svc
    namespace: etx-app-dev
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
    syncOptions:
      - CreateNamespace=false
EOF
```

The application continuously watches the `environments/dev` directory in the GitOps repository and automatically synchronizes any changes to the OpenShift cluster.



`CreateNamespace=false` is specified because the `etx-app-dev` namespace already exists.

Verify the Application Synchronization

Confirm that Argo CD has successfully synchronized the application.

1. Check the synchronization status:

   ```
   # Argo CD application status
   oc get application etx-app-dev -n etx-app-dev \
     -o jsonpath='{.status.sync.status}{"\n"}'
   ```

   The command should return:

   ```
   Synced
   ```

2. Confirm application pods are **running**:

   ```
   oc get pods -n etx-app-dev -l app=etx-app-base-app
   ```

Verify the Application Deployment

1. Verify the application is running:

```
oc get route etx-app-base-app -n etx-app-dev -o jsonpath='{.spec.host}{"\n"}'
```

Open the route URL in your browser to confirm the application is accessible.

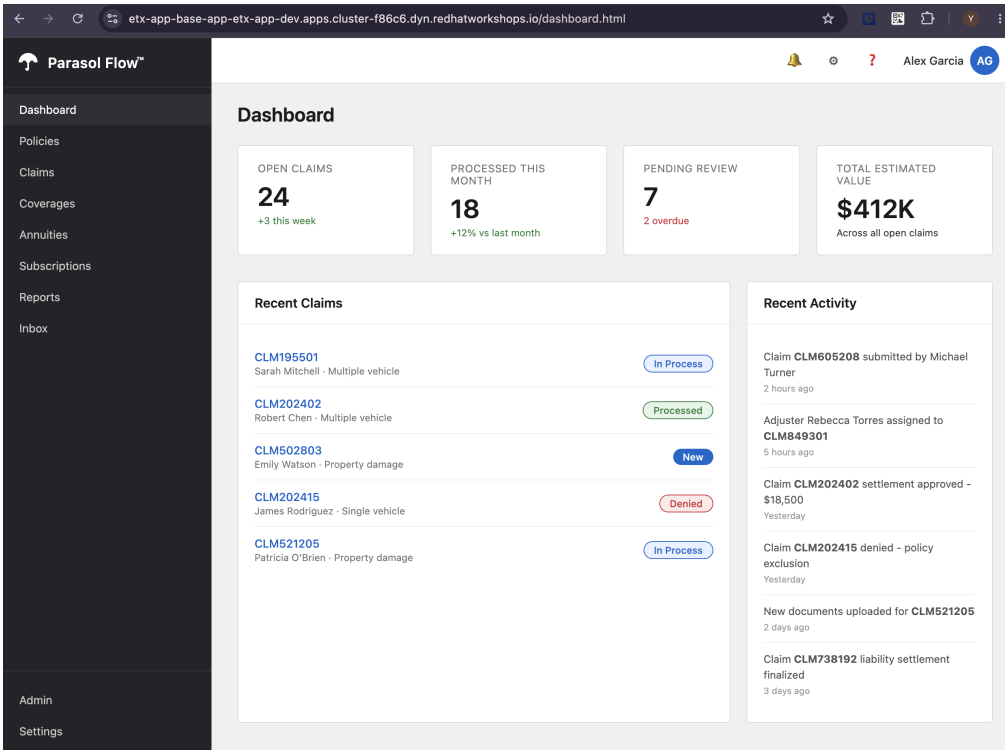


Figure 17. Parasol Application(Click the image to view it in full size in a new browser tab)

2. Verify that the argocd application status is **Synced**.

```
oc get route argocd-server -n etx-app-dev -o jsonpath='https://{.spec.host}{"\n"}'
```

Open the route URL in your browser to confirm the application is accessible.

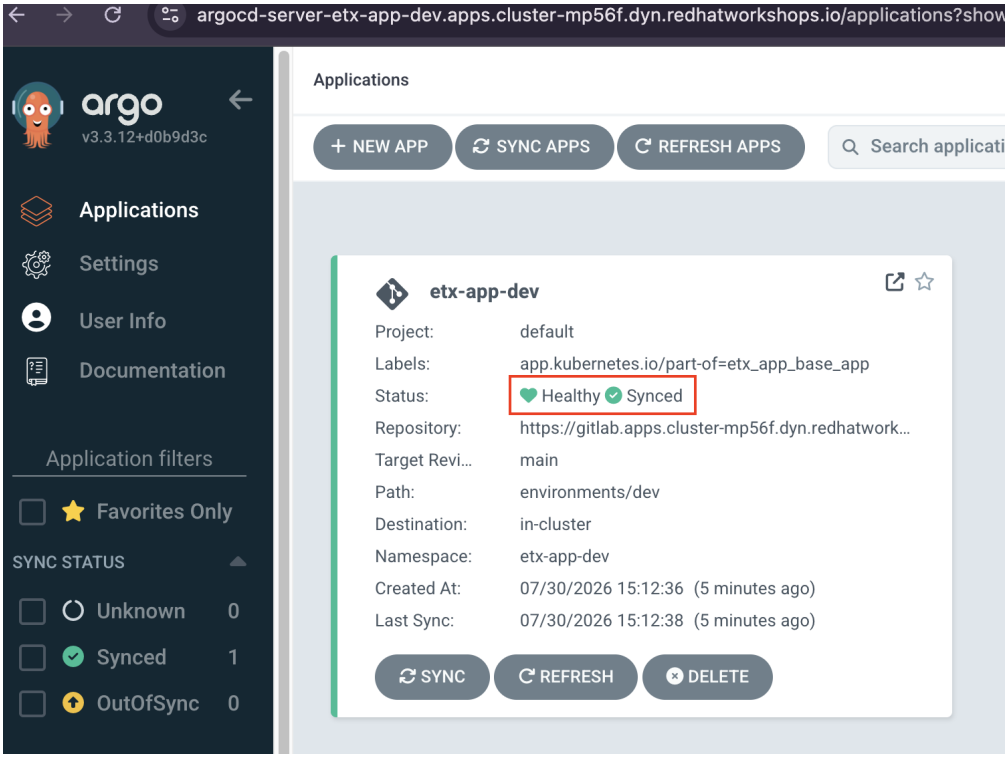


Figure 18. Argo CD dashboard(Click the image to view it in full size in a new browser tab)

Lab 9: Verify Observability

In this lab, you'll verify that OpenTelemetry auto-instrumentation is working correctly for the deployed application. You'll confirm that the OpenTelemetry Java agent has been injected into the application pod and that telemetry data is being collected by the OpenTelemetry Collector.

By the end of this lab, you will have:

- Verified that OpenTelemetry auto-instrumentation is enabled for the application
- Confirmed that the OpenTelemetry Java agent was injected into the application pod
- Verified that trace data is being collected by the OpenTelemetry Collector

Verify OpenTelemetry Auto-Instrumentation

In this exercise, you'll confirm that the OpenTelemetry Operator has automatically injected the Java agent into the application deployed through Argo CD.

1. Verify that the application pod contains the OpenTelemetry auto-instrumentation init container:

```
oc get pod -n etx-app-dev \
  -l app=etx-app-base-app \
  -o jsonpath='{.items[0].spec.initContainers[*].name}{"\n"}'
```

The output should include:

```
opentelemetry-auto-instrumentation
```

The presence of this init container confirms that the OpenTelemetry Operator has automatically injected the Java agent into the application pod.

Verify Trace Collection

Next, verify that the application is sending telemetry data to the OpenTelemetry Collector.

1. View the recent logs from the OpenTelemetry Collector:

```
oc logs -n etx-app-dev \
  -l app.kubernetes.io/name=otel-collector-collector \
  --tail=50
```

Review the output and confirm that trace data is being received and processed by the collector.

Once traces appear in the collector logs, you have verified that OpenTelemetry auto-instrumentation is successfully collecting telemetry from the deployed application.

Lab 10: CI to GitOps

In this lab, you'll connect your Tekton CI pipeline to your GitOps repository so that every successful build automatically updates the application image referenced by Argo CD.

By the end of this lab, you will have:

- Created a Tekton Task that updates the GitOps repository with a new image tag
- Extended the CI pipeline to invoke the GitOps update task after a successful image build
- Automated the flow from code commit to GitOps repository update

Create the GitOps Update Task

In this exercise, you'll create a Tekton Task that clones the GitOps repository, updates the application image tag, commits the change, and pushes it back to GitLab.

1. Open a terminal in your OpenShift Dev Spaces workspace or launch the **Web Terminal** from the OpenShift console.
2. Create the Task:

```
oc apply -f - <<'EOF'
apiVersion: tekton.dev/v1
kind: Task
metadata:
  name: update-gitops-repo
  namespace: etx-app-dev
spec:
  params:
    - name: gitops-repo-url
```

```
      type: string
    - name: image-tag
      type: string
    - name: environment
      type: string
      default: dev
  workspaces:
    - name: source
  steps:
    - name: update-image
      image: registry.access.redhat.com/ubi9/ubi:latest
      script: |
        #!/bin/bash
        set -e

        # Install git
        dnf install -y git && dnf clean all

        REPO_URL="$(params.gitops-repo-url)"
        GITLAB_HOST=$(echo "$REPO_URL" | sed -E 's|https?://([^/]+)/.*|\1|')

        git config --global url."https://etx-platform-
bot:openshift@${GITLAB_HOST}/".insteadOf "https://${GITLAB_HOST}/"

        git clone "$REPO_URL" /tmp/gitops
        cd /tmp/gitops/environments/$(params.environment)

        # Update the image tag in the deployment manifest
        sed -i "s|image:.*etx-app-base-app:.*|image: image-registry.openshift-
image-registry.svc:5000/etx-app-dev/etx-app-base-app:$(params.image-tag)|"
deployment.yaml

        git config user.email "pipeline@openshift.local"
        git config user.name "Tekton Pipeline"
        git add .
        git commit -m "Update etx-app-base-app image to $(params.image-tag)"
        git push
EOF
```

The Task updates the deployment manifest in the GitOps repository with the newly built image tag and pushes the change back to GitLab.

Update the CI Pipeline

Next, extend the existing CI pipeline so it automatically executes the GitOps update task after the container image has been built successfully.

1. Retrieve the GitLab URL:

```
GITLAB_URL="https://$(oc get route \
```

```
  -n gitlab \
  -l app=webservice \
  -o jsonpath='{.items[0].spec.host}')"

echo "${GITLAB_URL}"
```

2. Edit the existing pipeline:

```
oc edit pipeline etx-app-base-app-ci -n etx-app-dev
```

3. Under the `spec.tasks` section, add the following task definition:

```
    - name: update-gitops
      runAfter: [build-image]
      taskRef:
        name: update-gitops-repo
      params:
        - name: gitops-repo-url
          value: SUBSTITUTE-GITLAB-URL-HERE/software-factory/etx-app-gitops.git
        - name: image-tag
          value: $(params.git-revision)
        - name: environment
          value: dev
      workspaces:
        - name: source
          workspace: shared-workspace
```



Replace `SUBSTITUTE-GITLAB-URL-HERE` with the GitLab URL retrieved earlier. If you're recreating the pipeline using a shell heredoc, you can use the `${GITLAB_URL}` variable instead.

Verify the Integration

You'll trigger the CI pipeline by making a code change and verify that the GitOps repository is updated automatically. You'll then confirm that Argo CD detects the change and deploys the new application version.

1. Open a terminal in your Dev Spaces workspace.
2. Make a small change to the application and commit it:

```
echo "====" >> README.md
git add -A
git commit -m "Verify CI to GitOps integration"
git push -u origin feature/add-devfile
```

3. Monitor the pipeline until it completes successfully:

```
oc get pipelineruns -n etx-app-dev --watch
```

Wait until the latest PipelineRun shows a status of `Succeeded`.

4. In Openshift console terminal, verify that the GitOps repository **etx-app-gitops** has been updated automatically:

```
GITLAB_URL="https://$(oc get route -n gitlab \
  -l app=webservice \
  -o jsonpath='{.items[0].spec.host}')"
git clone ${GITLAB_URL}/software-factory/etx-app-gitops.git
git pull
grep "image:" environments/dev/deployment.yaml
```

Sample output:

```
image: image-registry.openshift-image-registry.svc:5000/etx-app-dev/etx-app-base-
app:30615ccd54c01ac19f28a0a71df611ff22dde254
```

The image tag should now match the Git commit SHA from the pipeline execution.

5. View the latest Git commit:

```
git log -1 --oneline
```

You should see a commit similar to:

```
1df86ef (HEAD -> main, origin/main, origin/HEAD) Update etx-app-base-app image to
30615ccd54c01ac19f28a0a71df611ff22dde254
```

6. Verify that Argo CD synchronized the change:

```
oc get applications -n openshift-gitops
```

Confirm that the `etx-app-dev` application reports `Synced` and `Healthy`.

7. Verify that the application is running the updated image:

```
oc describe deployment etx-app-base-app -n etx-app-dev | grep Image:
```

Sample Output:

```
Image:      image-registry.openshift-image-registry.svc:5000/etx-app-dev/etx-app-
base-app:30615ccd54c01ac19f28a0a71df611ff22dde254
```

The deployment should reference the same image tag that was committed to the GitOps repository.



The complete deployment flow is now fully automated:

- A code commit triggers the Tekton CI pipeline.
- Tekton builds and pushes a new container image.
- The pipeline updates the GitOps repository with the new image tag.
- A new commit is pushed to GitLab automatically.
- Argo CD detects the Git change and synchronizes the application.
- OpenShift deploys the updated application without any manual intervention.

Lab Summary

You have completed the GitOps deployment workflow. Your application platform now provides:

- A dedicated Argo CD instance integrated with Keycloak Single Sign-On
- A GitOps repository containing environment-specific Kubernetes manifests
- Automated application deployment and synchronization with self-healing enabled
- OpenTelemetry auto-instrumentation for application observability
- End-to-end CI/CD integration, where successful Tekton builds automatically update the GitOps repository and trigger application deployment through Argo CD

Application Promotion and GitOps Deployment Workflow



What's in this section?

In this section, you'll learn how modern application platforms promote applications across environments using a **secure, GitOps-based delivery workflow**.

You'll explore how container images move from development to staging without rebuilding, how Git remains the source of truth for deployment configuration, and how tools such as **OpenShift Pipelines, Quay, Vault, and Argo CD** enable automated, secure, and repeatable application delivery.

Throughout this section, you'll build an application promotion workflow that provides:

- Automated image promotion between environments
- Secure registry credential management
- GitOps-based deployment synchronization
- Multi-cluster application delivery
- Reliable rollback capabilities using Git history and validated images

Lab Pre-requisites

In this lab, you'll prepare the environment required for multi-cluster application promotion. You'll enroll a runtime cluster with Argo CD, provision the required supporting services, and understand how application promotion moves validated container images from the development environment to staging.



- Factory cluster / development cluster: **Service Enablement: App Platform Software Factory**
- Runtime cluster / staging cluster: **Service Enablement: App Platform Runtime**
- Make sure you have provisioned **runtime cluster** before starting the labs in this section.

By the end of this lab, you will have:

- Reviewed the multi-cluster application promotion workflow
- Verified the prerequisites for multi-cluster application promotion
- Enrolled a runtime cluster with the factory Argo CD instance
- Provisioned the staging environment and supporting platform services
- Verified that the runtime cluster is registered successfully

Review the Promotion Architecture

In the previous exercises, you built a CI pipeline that automatically builds container images and deploys them to the development environment.

In this lab, you'll prepare the infrastructure required to promote validated application images to a staging environment running on a separate OpenShift cluster.

The promotion workflow performs the following actions:

- Copies container images from the OpenShift internal registry to Quay for cross-cluster access.
- Updates the GitOps repository with the new staging image tag.
- Allows Argo CD to synchronize the updated manifests to the runtime cluster.

Understand the Promotion Workflow

Unlike the development environment, where every successful build is deployed automatically, promotion to staging is an intentional action initiated by the development team.

The promotion workflow consists of the following steps:

Table 1. Promotion Workflow

Step	Component	Action
1	Developer	Approves the promotion by manually triggering the promotion pipeline.
2	GitLab	Starts the promotion pipeline.
3	Promotion Pipeline	Pulls the application image from the OpenShift internal registry using the image SHA.
4	Promotion Pipeline	Pushes the image to Quay using the `stable` tag.
5	Promotion Pipeline	Updates the `environments/stage/deployment.yaml` manifest in the GitOps repository.
6	Argo CD	Detects the updated GitOps repository.
7	Argo CD	Synchronizes the updated manifests and deploys the application to the staging cluster.

The promotion workflow follows these principles:

- **Development** uses images stored in the OpenShift internal registry for fast local deployments.

- **Staging** uses images stored in Quay so they can be accessed by multiple OpenShift clusters.
- **Promotion is explicit**, allowing developers to decide when a validated application is ready for staging.
- **GitOps ensures deployments remain version-controlled**, with every staging deployment driven by changes committed to the GitOps repository.

Verify the Prerequisites

Before continuing, ensure you have completed the following exercises:

- Continuous Integration with OpenShift Pipelines
- GitOps Continuous Deployment

You should also have:

- A working CI pipeline that builds and pushes application images
- Argo CD managing the application deployment
- Access to Quay using Keycloak Single Sign-On (`platform-admin` or `platform-developer`)
- A runtime OpenShift cluster provisioned through RHDP

Lab 11: Getting started

The promotion workflow requires two OpenShift clusters:

- **Factory Cluster (Service Enablement: App Platform Software Factory)** – Hosts the Software Factory, CI pipelines, and Argo CD.
- **Runtime Cluster (Service Enablement: App Platform Runtime)** – Hosts the staging and production application environments.

In this exercise, you'll register the runtime cluster with the Argo CD instance running on the factory cluster.

Enroll the Runtime Cluster

1. Download the enrollment script:

```
curl -o enroll-secondary-cluster.sh \
  https://raw.githubusercontent.com/rh-etx-app-
platform/etx_app_tooling/main/scripts/enroll-secondary-cluster.sh

chmod +x enroll-secondary-cluster.sh
```

2. Substitute the placeholder values (`RUNTIME-GUID`, `RUNTIME_CLUSTER_ADMIN_PASSWORD`, `FACTORY-GUID`, and `FACTORY_CLUSTER_ADMIN_PASSWORD`) with the information provided in your **RHDP lab environment**, then run the following commands to configure the environment variables used

throughout the remaining labs.

```
export RUNTIME_API="https://api.cluster-<RUNTIME-GUID>.dyn.redhatworkshops.io:6443"
export RUNTIME_USER="admin"
export RUNTIME_PASSWORD="<RUNTIME_CLUSTER_ADMIN_PASSWORD>"

export FACTORY_API="https://api.cluster-<FACTORY-GUID>.dyn.redhatworkshops.io:6443"
export FACTORY_USER="admin"
export FACTORY_PASSWORD="<FACTORY_CLUSTER_ADMIN_PASSWORD>"
```

3. Run the enrollment script:

```
./enroll-secondary-cluster.sh
```

The script automatically performs the following tasks:

- Creates the required ServiceAccount and permissions on the runtime cluster.
- Creates the staging and production namespaces.
- Installs the required platform operators.
- Deploys PostgreSQL, Kafka, OpenTelemetry Collector, and Java auto-instrumentation.
- Registers the runtime cluster with the factory Argo CD instance.

When the script completes successfully, you should see an **Enrollment Complete** message.

Verify the Runtime Cluster Registration

Confirm that the runtime cluster has been registered with Argo CD.

Run the following command:

```
oc get secret -n etx-app-dev \
  -l argocd.argoproj.io/secret-type=cluster \
  -o json | \
jq -r '.items[] | .metadata.name + " - " + (.data.name | @base64d)'
```

Verify that the output includes both clusters:

```
argocd-default-cluster-config - in-cluster
cluster-staging - staging
```

If both entries are displayed, the runtime cluster has been enrolled successfully and is ready to receive application deployments through Argo CD.

In the next lab, you'll build the promotion pipeline that automatically promotes validated

application images from the factory cluster to the staging environment.

Lab 12: Configure Quay

In this exercise, you'll prepare Quay for the image promotion workflow by creating the repository used by the promotion pipeline and storing robot account credentials securely in Vault.

By the end of this exercise, you will have:

- Created the `etx-app` organization and `etx-app-base-app` repository in Quay
- Created a robot account for non-interactive image promotion
- Stored the Quay robot credentials in Vault for use by the promotion pipeline

The promotion pipeline pushes images to Quay at `etx-app/etx-app-base-app` and retrieves authentication credentials from Vault at `secret/registry/quay`.



Interactive users authenticate to Quay using Keycloak Single Sign-On, while CI/CD pipelines use robot accounts for automated, non-interactive authentication.

Create the Quay Organization and Repository

Create the Quay organization and repository that will store promoted application images.

1. Open the **Quay** URL from your **RHDP lab environment**.

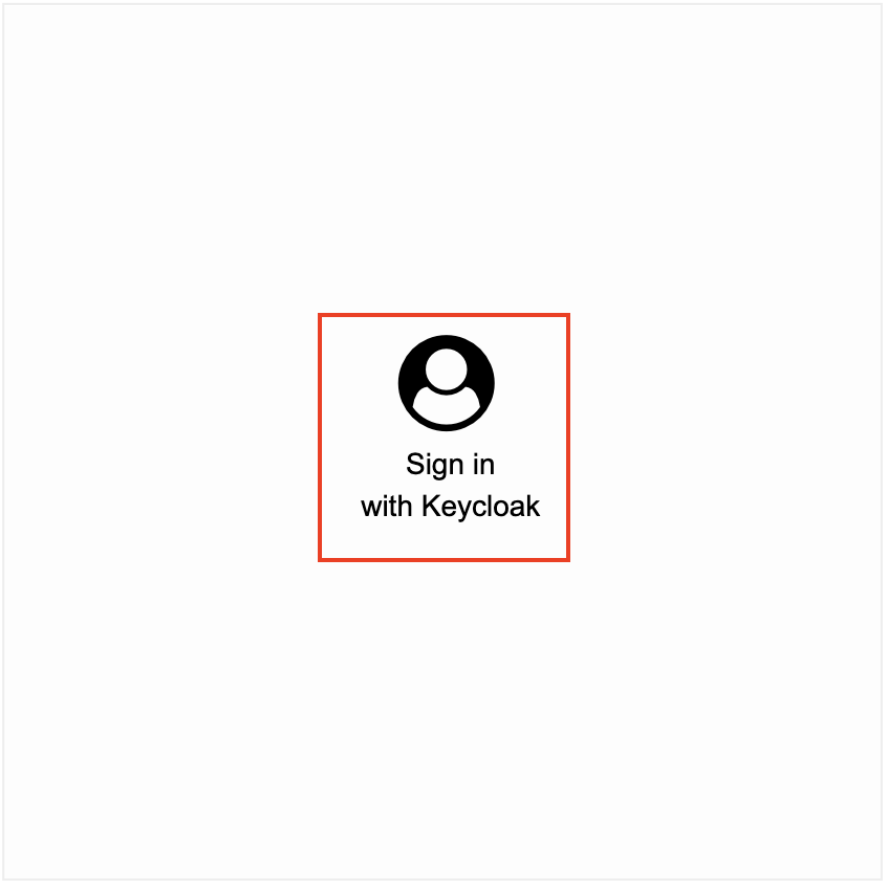


Figure 19. Quay Login Page(Click the image to view it in full size in a new browser tab)

2. Sign in using **ETX Keycloak Single Sign-On** with one of the lab accounts:
   - `platform-admin` / `openshift`
   - `platform-developer` / `openshift`
3. If prompted, remove any hyphens (`-`) from the suggested username and click **[Confirm Username]**.

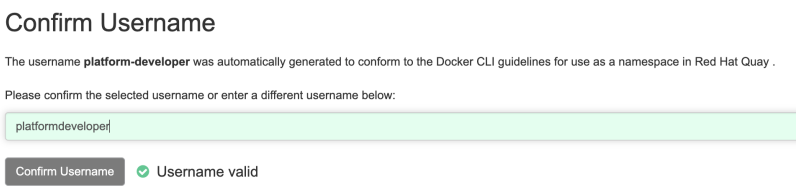


Figure 20. Validate Quay Username

4. Click **[Create New Organization]**.

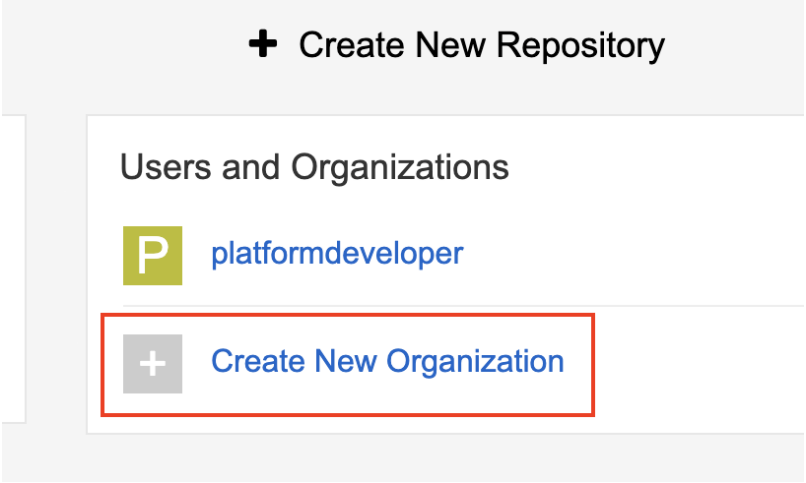


Figure 21. Create Organization

5. Create an organization named `etx-app`.

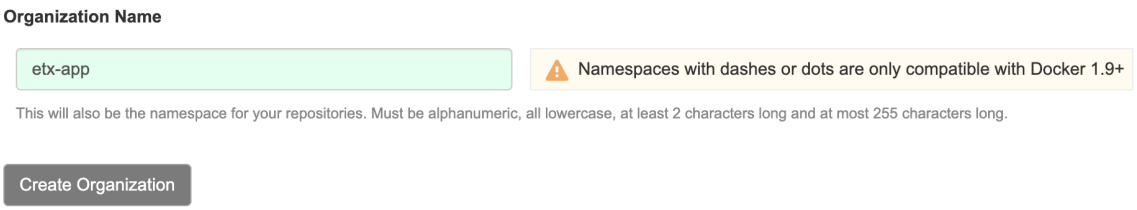


Figure 22. Create the Organization

6. Inside the organization, click **[Create New Repository]**
7. Configure the repository and click **[Create Public Repository]**.:
   - **Repository Name:** `etx-app-base-app`
   - **Visibility:** Public

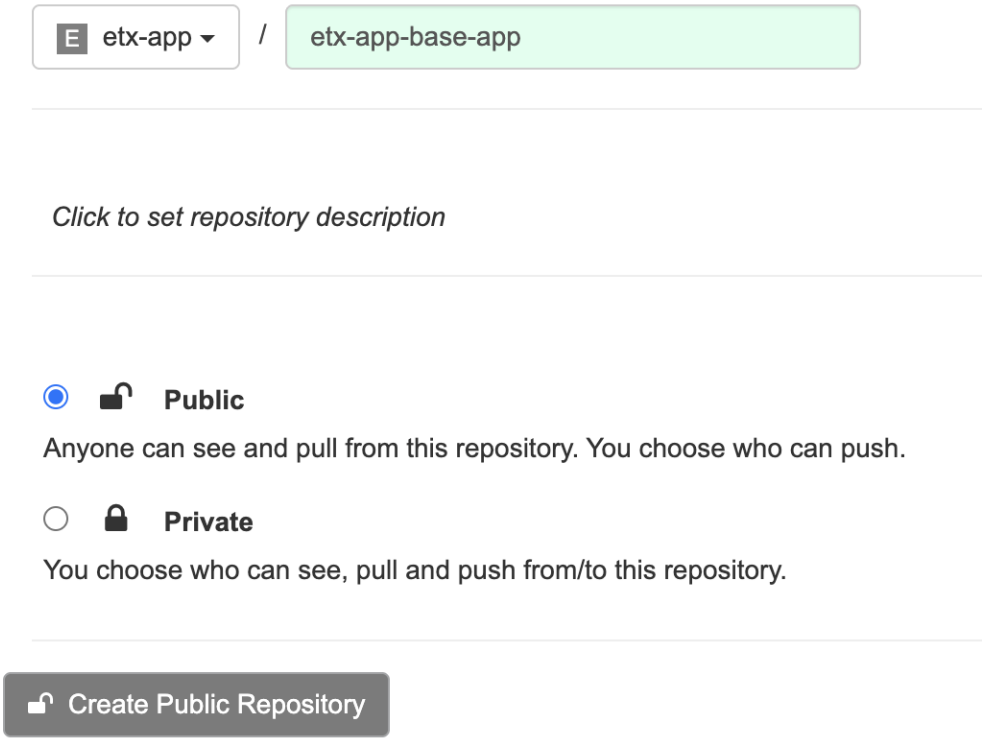


Figure 23. Create Repository



The organization and repository names must match the destination referenced later by the promotion pipeline: `etx-app/etx-app-base-app`.



Although the repository is public, anonymous image pulls are disabled in this Quay deployment. Runtime clusters must authenticate using an image pull secret.

Create a Robot Account

The promotion pipeline requires a robot account to authenticate to Quay without using interactive Keycloak credentials.

1. From the `etx-app` organization, click the **Settings** icon.

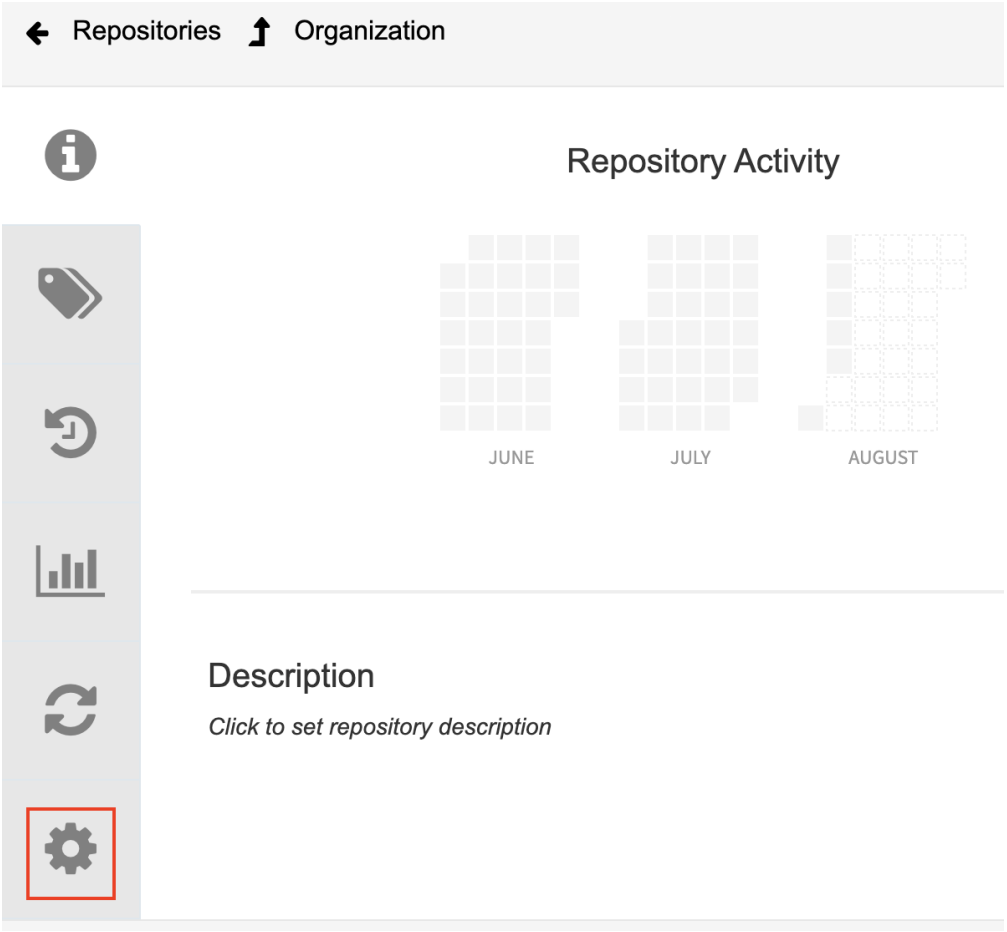


Figure 24. Organization Settings

2. Under **User and Robot Permissions**, open the dropdown and click **[Create Robot Account]**.

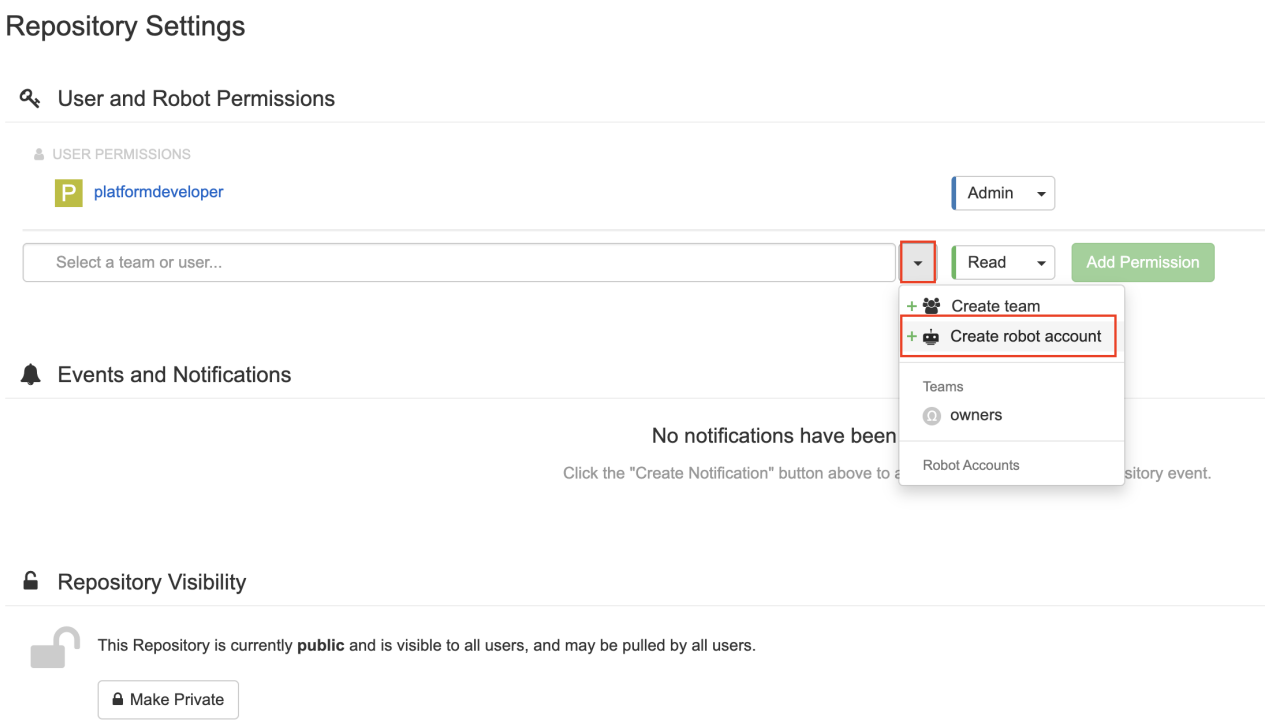


Figure 25. Create Robot Account

3. Create a robot account named `promoter` and click **[Create robot account]**.

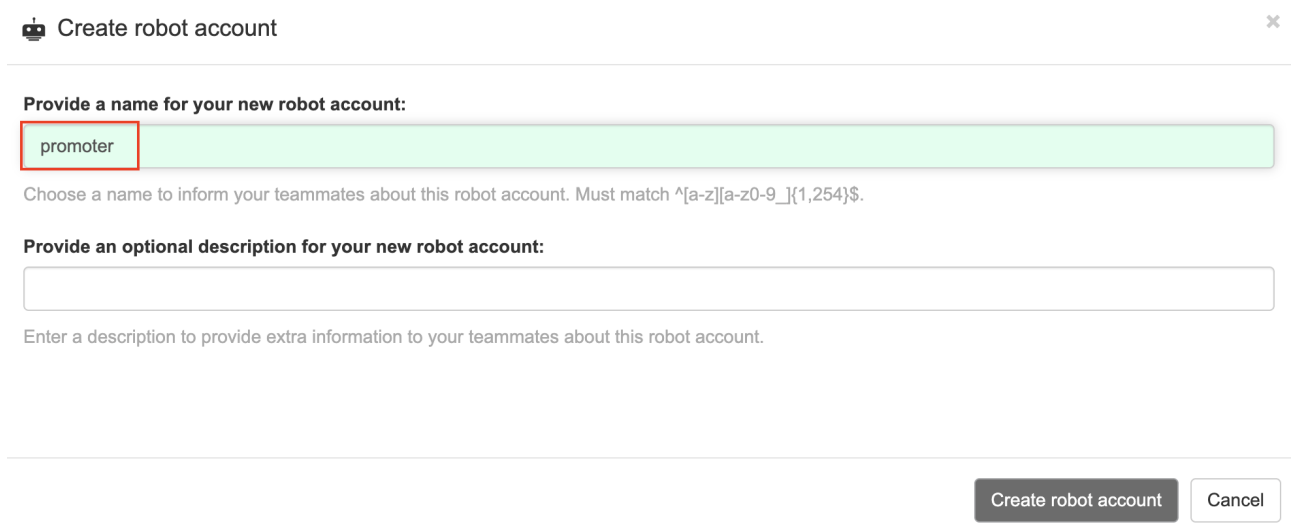


Figure 26. Robot Account Name

4. Grant the robot account **Write** permission to the repository.

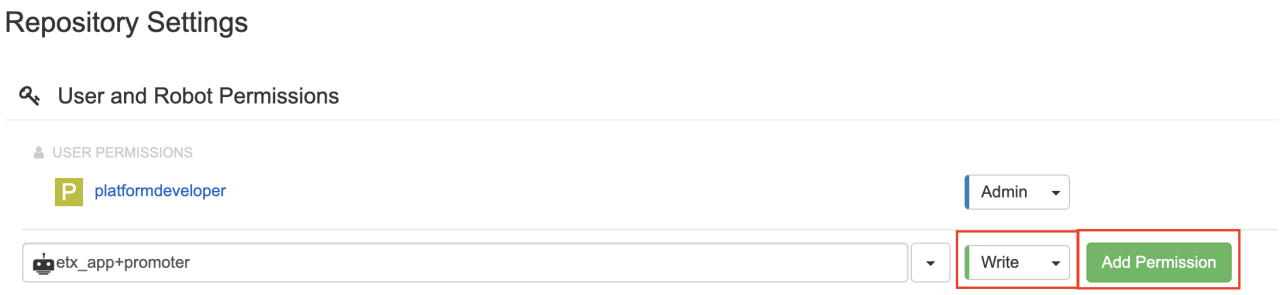


Figure 27. Assign Write Permission

5. Open the newly created robot account by clicking **[etx-app+promoter]**

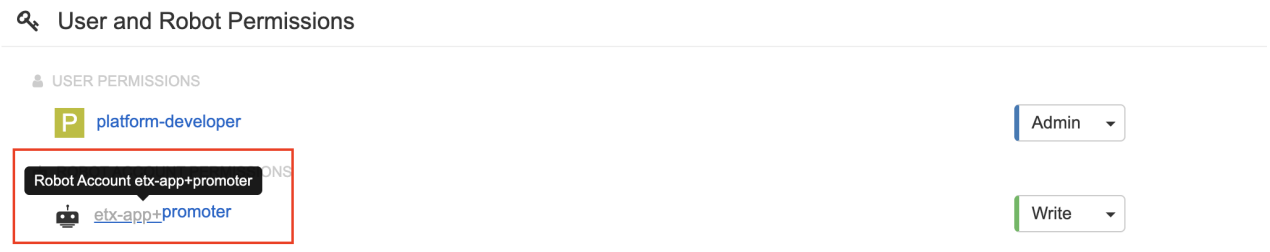


Figure 28. Robot Account

6. Copy the generated credentials:
   - **Username:** `etx-app+promoter`
   - **Token:** Generated robot token

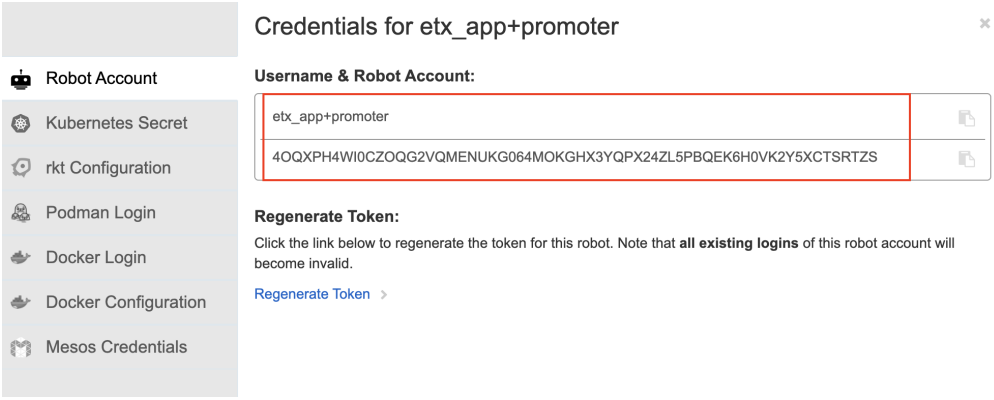


Figure 29. Robot Account Credentials



Save the robot token before leaving the page. It will be required when creating the Vault secret.

Store Quay Credentials in Vault

Store the Quay robot account credentials in Vault so the promotion pipeline can retrieve them securely at runtime.

1. Retrieve the Quay hostname:

```
oc get route -n quay \
  -l quay-component=quay-app-route \
  -o jsonpath='{.items[0].spec.host}' && echo
```

2. Retrieve the Vault root token:

```
oc get secret etx-vault-bootstrap \
  -n vault \
  -o jsonpath='{.data.root-token}' | base64 -d && echo
```

3. Open the **Vault** URL from your lab environment.
4. Sign in using the **Token** authentication method and paste the token retrieved above.

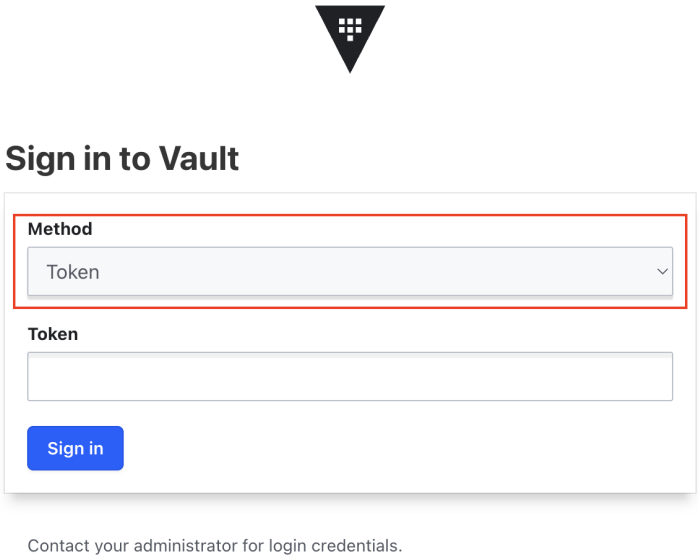


Figure 30. Sign in to Vault

5. From the Vault dashboard, open the `secret/` secrets engine.

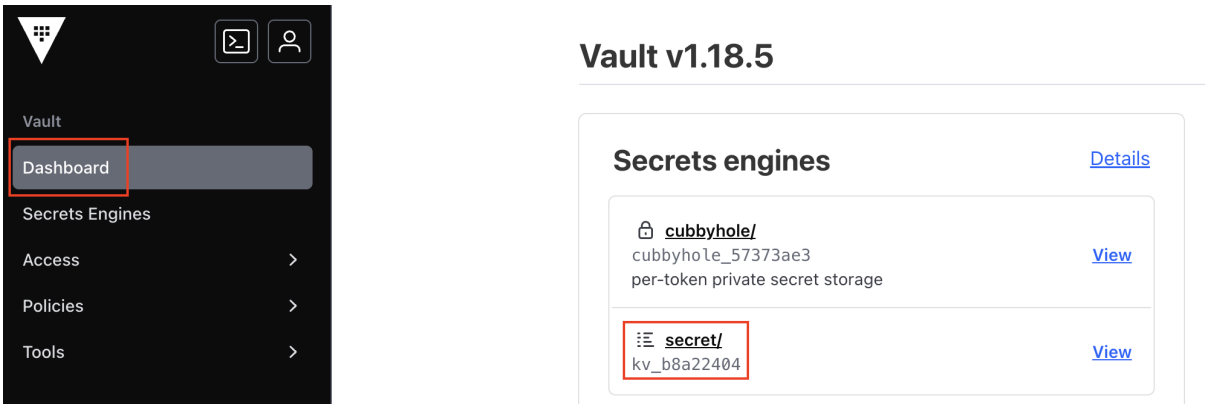


Figure 31. Secret Engine

6. Click **[Create secret]**.

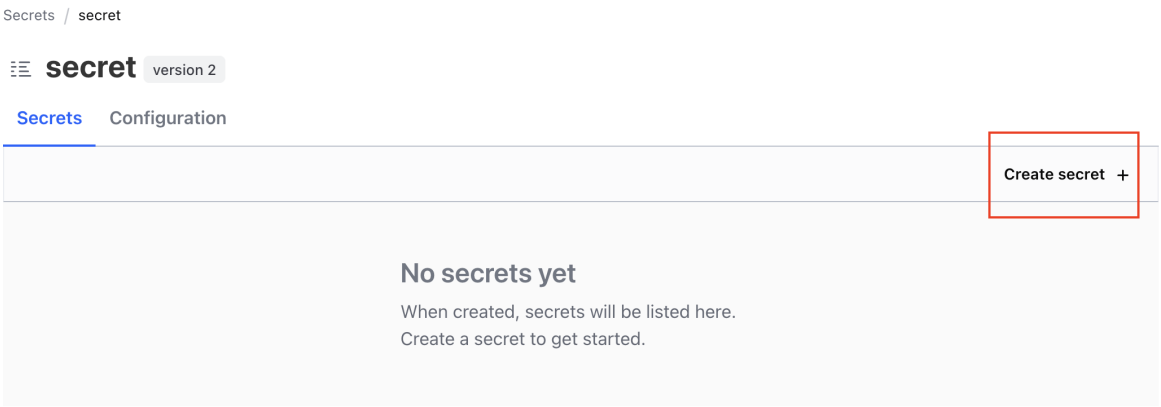


Figure 32. Create Secret

7. Create a secret with the path:

```
registry/quay
```

8. Add the following key-value pairs:
   - **server:** Quay hostname returned by the earlier command in Step 1 (without `https://`)
   - **username:** `etx-app+promoter`
   - **password:** Robot token copied from Quay

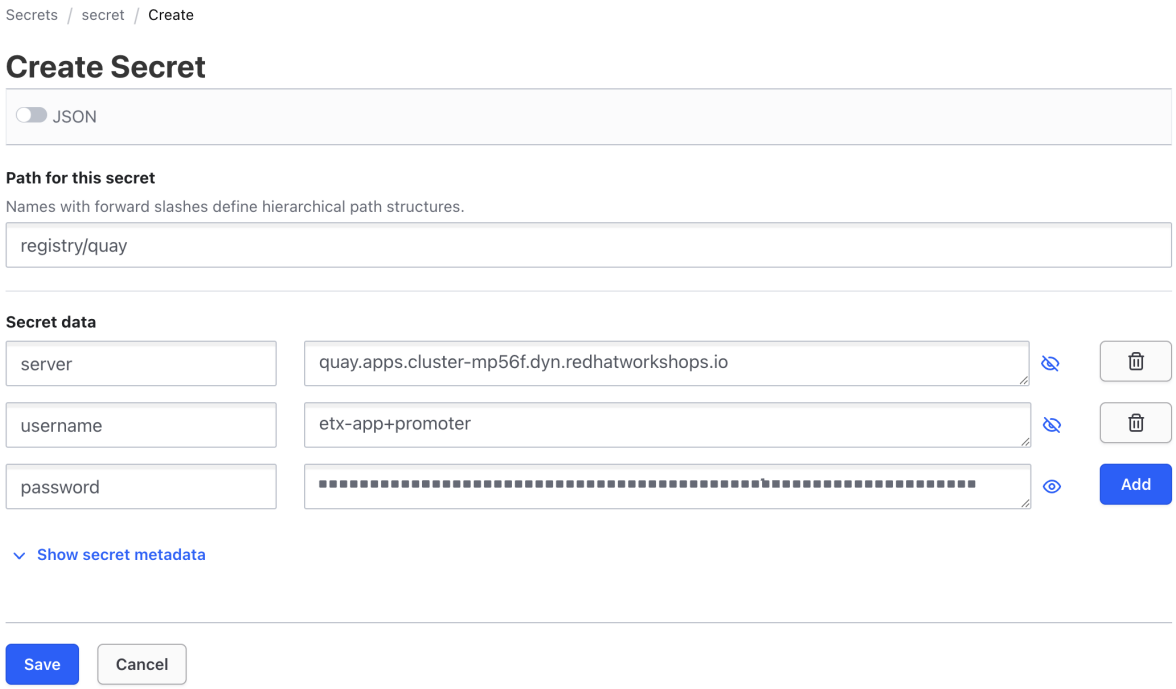


Figure 33. Create Vault Secret

9. Click **[Save]**.

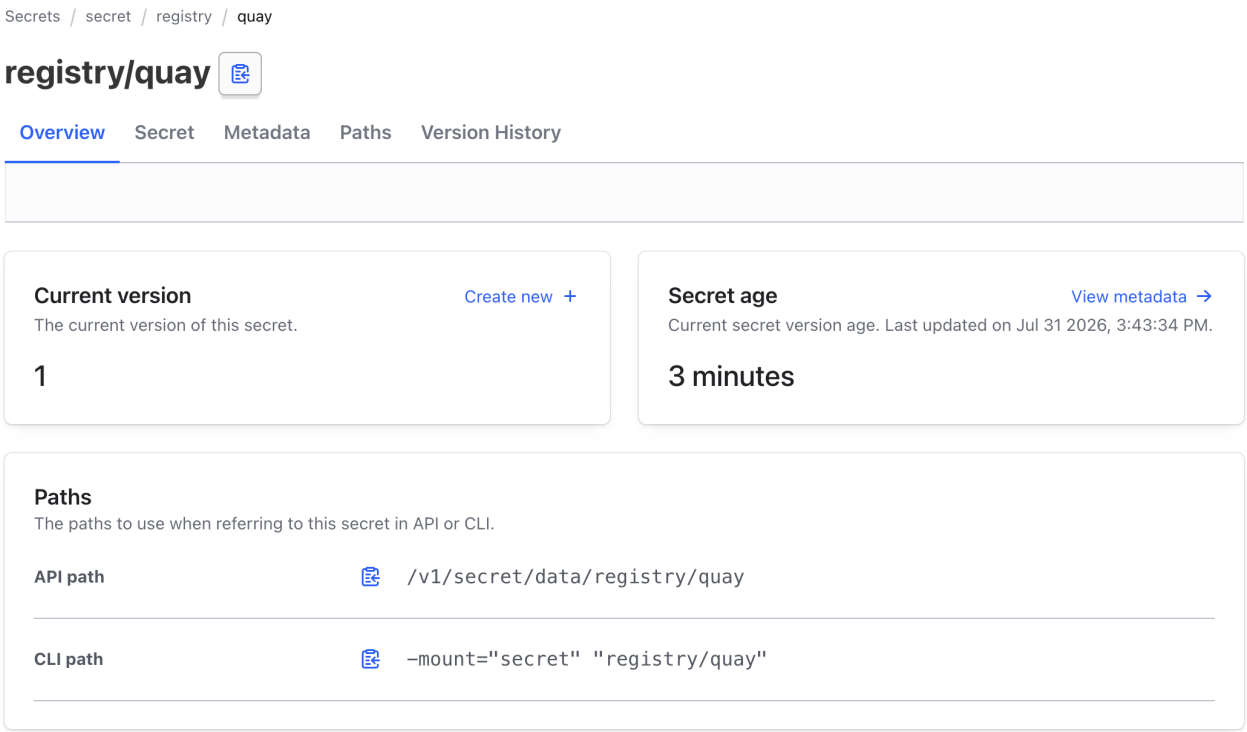


Figure 34. Completed Secret



The promotion pipeline authenticates to Vault using Kubernetes authentication and retrieves these credentials automatically during execution. No Quay credentials are stored directly in the pipeline definition.

Lab 13: Stage Deployment

In this lab, you'll prepare the staging environment for multi-cluster deployments by creating stage-specific GitOps manifests and configuring the runtime cluster to securely pull application images from Quay.

By the end of this lab, you will have:

- Created GitOps manifests for the staging environment
- Configured the runtime cluster to authenticate with Quay using an image pull secret
- Updated the staging deployment to use Quay-hosted images
- Committed and pushed the staging manifests to the GitOps repository

Prepare the Stage Environment

In this exercise, you'll create a staging environment in the GitOps repository based on the existing development manifests.

1. Clone the GitOps repository:

```
GITLAB_URL="https://$(oc get route \
  -n gitlab \
```

```
  -l app=webservice \
  -o jsonpath='{.items[0].spec.host}')"

git clone ${GITLAB_URL}/software-factory/etx-app-gitops.git
cd etx-app-gitops
mkdir -p environments/stage
```

2. Copy the development manifests as the starting point for the staging environment.

```
cp environments/dev/*.yaml environments/stage/
```

3. Retrieve the Quay registry URL that will host promoted application images.

```
QUAY_URL=$(oc get route -n quay -l quay-component=quay-app-route -o
jsonpath='{.items[0].spec.host}')
echo "Using Quay URL: $QUAY_URL"
```

Configure Image Pull Access

The runtime cluster must authenticate with Quay before it can pull promoted container images.

1. Log in to the **Service Enablement: App Platform Runtime** cluster.

```
oc login -u $RUNTIME_USER -p $RUNTIME_PASSWORD $RUNTIME_API --insecure-skip-tls
-verify
```



Make sure you have properly set the `RUNTIME_USER`, `RUNTIME_PASSWORD`, and `RUNTIME_API` values as described in Step 2 of Lab 11: Enroll the Runtime Cluster.

2. Create a docker-registry secret in `etx-app-staging` on the runtime cluster using the robot account credentials. Replace **$ROBOT_TOKEN** with the etx-app+promoter token:

```
oc create secret docker-registry quay-pull-secret \
  -n etx-app-staging \
  --docker-server=$QUAY_URL \
  --docker-username='etx-app+promoter' \
  --docker-password=$ROBOT_TOKEN
```

3. Switch back to the factory cluster using the credentials provided by RHDP lab environment page.

```
oc login $FACTORY_API -u $FACTORY_USER -p $FACTORY_PASSWORD --insecure-skip-tls
-verify
```



Make sure you have properly set the `FACTORY_USER`, `FACTORY_PASSWORD`, and `FACTORY_API` values as described in Step 2 of Lab 11: Enroll the Runtime Cluster.

Create the Stage Deployment Manifest

Create a deployment manifest that references the staging namespace and Quay-hosted application image.

1. Navigate to the **etx-app-gitops** repository that you cloned previously or freshly clone the **etx-app-gitops** repository locally:

```
GITLAB_URL="https://$(oc get route -n gitlab -l app=webservice -o
jsonpath='{.items[0].spec.host}')"

git clone ${GITLAB_URL}/software-factory/etx-app-gitops.git
cd etx-app-gitops
```

2. Replace the existing deployment manifest with the following configuration.

```
cat > environments/stage/deployment.yaml <<EOF
apiVersion: apps/v1
kind: Deployment
metadata:
  name: etx-app-base-app
  namespace: etx-app-staging
  labels:
    app: etx-app-base-app
  annotations:
    instrumentation.opentelemetry.io/inject-java: "true"
spec:
  replicas: 2
  selector:
    matchLabels:
      app: etx-app-base-app
  template:
    metadata:
      labels:
        app: etx-app-base-app
      annotations:
        instrumentation.opentelemetry.io/inject-java: "true"
    spec:
      imagePullSecrets:
        - name: quay-pull-secret
      containers:
        - name: etx-app-base-app
          image: ${QUAY_URL}/etx-app/etx-app-base-app:stable
          ports:
            - containerPort: 8080
```

```
          resources:
            requests:
              cpu: 500m
              memory: 1Gi
            limits:
              cpu: "2"
              memory: 2Gi
          env:
            - name: POSTGRESQL_HOST
              value: "parasol-db.etx-app-staging.svc.cluster.local"
            - name: POSTGRESQL_DATABASE
              value: "parasol"
            - name: POSTGRESQL_USER
              value: "parasol"
            - name: POSTGRESQL_PASSWORD
              value: "parasol"
            - name: QUARKUS_HIBERNATE_ORM_DATABASE_GENERATION
              value: update
EOF
```

Compared to the development environment, the staging deployment includes several production-oriented changes:

- Deploys to the `etx-app-staging` namespace
- Runs two application replicas for improved availability
- Pulls images from the Quay registry instead of the OpenShift internal registry
- Uses the `quay-pull-secret` to authenticate with Quay

Update and Commit the Stage Manifests

Update the remaining manifests to reference the staging namespace and commit the changes to the GitOps repository.

1. Update the Service and Route manifests.

```
sed -i 's/namespace: etx-app-dev/namespace: etx-app-staging/' \
  environments/stage/service.yaml \
  environments/stage/route.yaml
```

2. Verify that all manifests reference the staging namespace.

```
grep namespace environments/stage/*.yaml
```

   Sample output:

```
environments/stage/deployment.yaml:  namespace: etx-app-staging
```

```
environments/stage/route.yaml:  namespace: etx-app-staging
environments/stage/service.yaml:  namespace: etx-app-staging
```

3. Commit and push the staging environment manifests.

```
git add -A
git commit -m "feat: add stage environment manifests"
git push origin main
```

Once the changes are pushed, the GitOps repository contains separate deployment definitions for both the development and staging environments, enabling Argo CD to manage each environment independently.

Lab 14: Image Promotion Pipeline

In this lab, you'll create the Tekton resources required to promote application images from the development environment to the staging environment. The promotion pipeline securely retrieves credentials from Vault, copies images to Quay, and updates the GitOps repository to trigger a deployment.

By the end of this lab, you will have:

- Created a workspace for the promotion pipeline
- Retrieved Quay credentials securely from Vault
- Created a task to copy images from the OpenShift internal registry to Quay
- Created a task to update the staging GitOps manifests automatically

Create the Promotion Pipeline Workspace

The promotion pipeline requires persistent storage for cloning and updating the GitOps repository.

Create a PersistentVolumeClaim for the promotion pipeline workspace.

```
oc apply -f - <<'EOF'
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: promotion-workspace-pvc
  namespace: etx-app-dev
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 2Gi
EOF
```

Create the Vault Credentials Task

The promotion pipeline retrieves the Quay robot account credentials you stored in Vault rather than embedding secrets directly in the pipeline.

1. Copy the **Vault URL** from your lab environment.
2. Create the `vault-fetch-quay-creds` task, make sure to substitute <YOUR-VAULT-URL> in below file with your environment's vault url.

```
oc apply -f - <<'EOF'
apiVersion: tekton.dev/v1
kind: Task
metadata:
  name: vault-fetch-quay-creds
  namespace: etx-app-dev
spec:
  results:
    - name: quay-server
    - name: quay-username
    - name: quay-password
  steps:
    - name: fetch
      image: registry.access.redhat.com/ubi9/ubi-minimal:latest
      script: |
        #!/bin/sh
        set -e

        # Install jq
        microdnf install -y jq && microdnf clean all

        # Authenticate to Vault using ServiceAccount token
        VAULT_ADDR=<YOUR-VAULT-URL>
        SA_TOKEN=$(cat /var/run/secrets/kubernetes.io/serviceaccount/token)

        VAULT_TOKEN=$(curl -s --request POST \
          "${VAULT_ADDR}/v1/auth/kubernetes/login" \
          --data '{"role":"pipeline","jwt":"'"${SA_TOKEN}"'"}' \
          | jq -r '.auth.client_token')

        # Fetch Quay credentials
        CREDS=$(curl -s --header "X-Vault-Token: ${VAULT_TOKEN}" \
          "${VAULT_ADDR}/v1/secret/data/registry/quay")

        echo "${CREDS}" | jq -r '.data.data.server' \
          | tr -d '\n' > $(results.quay-server.path)
        echo "${CREDS}" | jq -r '.data.data.username' \
          | tr -d '\n' > $(results.quay-username.path)
        echo "${CREDS}" | jq -r '.data.data.password' \
          | tr -d '\n' > $(results.quay-password.path)
```

```
EOF
```



The task authenticates to Vault using the pipeline ServiceAccount and Kubernetes authentication. Quay credentials are never stored in the pipeline definition.

Create the Image Promotion Task

After retrieving the credentials, the promotion pipeline copies the application image from the OpenShift internal registry to Quay using Skopeo.

Create the `copy-image-to-quay` task.

```
oc apply -f - <<'EOF'
apiVersion: tekton.dev/v1
kind: Task
metadata:
  name: copy-image-to-quay
  namespace: etx-app-dev
spec:
  params:
    - name: source-image
      type: string
      description: Source image URL (internal registry)
    - name: dest-image
      type: string
      description: Destination image URL (Quay)
    - name: dest-username
      type: string
    - name: dest-password
      type: string
  steps:
    - name: copy
      image: quay.io/skopeo/stable:latest
      script: |
        #!/bin/bash
        set -e

        echo "Copying image from internal registry to Quay..."
        echo "Source: $(params.source-image)"
        echo "Destination: $(params.dest-image)"

        # Authenticate to the OpenShift internal registry with the
        # pipeline ServiceAccount token (destination uses Quay robot creds)
        SRC_TOKEN=$(cat /var/run/secrets/kubernetes.io/serviceaccount/token)

        # Copy image using skopeo (single line to avoid YAML escaping issues)
        skopeo copy --src-tls-verify=false --dest-tls-verify=false --src
-creds="pipeline:${SRC_TOKEN}" --dest-creds="$(params.dest-username):$(params.dest
```

```
-password)" "docker://$(params.source-image)" "docker://$(params.dest-image)"

        echo "Image copied successfully!"
EOF
```



Skopeo performs a registry-to-registry image copy without pulling the container image locally, making promotions faster and more efficient.

Create the GitOps Update Task

The final task updates the staging deployment manifest in the GitOps repository with the newly promoted image. Once the change is committed, Argo CD automatically synchronizes the staging environment.

Create the `update-gitops-staging` task.

```
oc apply -f - <<'EOF'
apiVersion: tekton.dev/v1
kind: Task
metadata:
  name: update-gitops-staging
  namespace: etx-app-dev
spec:
  params:
    - name: gitops-repo-url
      type: string
    - name: image-url
      type: string
      description: Full image URL to update in staging manifest
    - name: git-username
      type: string
      default: "Tekton Pipeline"
    - name: git-email
      type: string
      default: "pipeline@openshift.local"
  workspaces:
    - name: source
  steps:
    - name: update
      image: registry.access.redhat.com/ubi9/ubi:latest
      script: |
        #!/bin/bash
        set -e

        # Install git
        dnf install -y git && dnf clean all

        REPO_URL="$(params.gitops-repo-url)"
        GITLAB_HOST=$(echo "$REPO_URL" | sed -E 's|https?://([^/]+)/.*|\1|')
```

```
        git config --global url."https://etx-platform-
bot:openshift@${GITLAB_HOST}/".insteadOf "https://${GITLAB_HOST}/"

        # Clone GitOps repo
        git clone "$REPO_URL" /workspace/gitops
        cd /workspace/gitops

        # Update the image in staging deployment
        sed -i "s|image:.*etx-app-base-app:.*|image: $(params.image-url)|" \
          environments/stage/deployment.yaml

        # Commit and push only when the image reference changed
        git config user.email "$(params.git-email)"
        git config user.name "$(params.git-username)"
        git add environments/stage/deployment.yaml
        if git diff --cached --quiet; then
          echo "No changes to commit - staging already references $(params.image-url)"
        else
          git commit -m "Promote to staging: $(params.image-url)"
          git push origin main
          echo "GitOps repository updated successfully!"
        fi
EOF
```

Once these tasks are created, you have all the building blocks required for the promotion pipeline. In the next lab, you'll assemble these tasks into a complete Tekton pipeline that automates image promotion from development to the staging environment.

Lab 15: Promotion Pipeline

In this lab, you'll assemble the promotion tasks into a complete Tekton pipeline and verify that it is ready to promote application images from the development environment to the staging environment.

By the end of this lab, you will have:

- Created a Tekton promotion pipeline
- Connected the pipeline tasks into a complete promotion workflow
- Configured the pipeline to copy images to Quay and update the GitOps repository
- Verified the pipeline configuration before running promotions

Create the Promotion Pipeline

In this exercise, you'll combine the tasks created in the previous lab into a single Tekton pipeline.

The promotion pipeline performs the following actions:

- Retrieves Quay credentials securely from Vault
- Copies the selected image from the OpenShift internal registry to Quay
- Updates the staging deployment manifest in the GitOps repository
- Triggers Argo CD to deploy the promoted image to the staging cluster

Copy the GITLAB_URL as it will required in next step.

```
export GITLAB_URL="https://$(oc get route -n gitlab -l app=webservice -o
jsonpath='{.items[0].spec.host}')"
echo $GITLAB_URL
```

1. Create the promotion pipeline, make sure to substitute <GITLAB_URL>.

```
oc apply -f - <<'EOF'
apiVersion: tekton.dev/v1
kind: Pipeline
metadata:
  name: etx-app-promotion
  namespace: etx-app-dev
spec:
  params:
    - name: source-image-tag
      type: string
      description: Image tag to promote (e.g., commit SHA)
    - name: dest-image-tag
      type: string
      default: stable
      description: Tag to use in Quay (e.g., stable, v1.0.0)
    - name: gitops-repo-url
      type: string
      default: <GITLAB_URL>/software-factory/etx-app-gitops.git
  workspaces:
    - name: shared-workspace
  tasks:
    - name: fetch-quay-creds
      taskRef:
        name: vault-fetch-quay-creds

    - name: copy-to-quay
      runAfter: [fetch-quay-creds]
      taskRef:
        name: copy-image-to-quay
      params:
        - name: source-image
          value: image-registry.openshift-image-registry.svc:5000/etx-app-dev/etx-
app-base-app:$(params.source-image-tag)
        - name: dest-image
          value: $(tasks.fetch-quay-creds.results.quay-server)/etx-app/etx-app-
```

```
base-app:$(params.dest-image-tag)
        - name: dest-username
          value: $(tasks.fetch-quay-creds.results.quay-username)
        - name: dest-password
          value: $(tasks.fetch-quay-creds.results.quay-password)

    - name: update-gitops
      runAfter: [copy-to-quay]
      taskRef:
        name: update-gitops-staging
      params:
        - name: gitops-repo-url
          value: $(params.gitops-repo-url)
        - name: image-url
          value: $(tasks.fetch-quay-creds.results.quay-server)/etx-app/etx-app-
base-app:$(params.dest-image-tag)
      workspaces:
        - name: source
          workspace: shared-workspace
EOF
```



The pipeline promotes an existing application image. It does not rebuild the application. Instead, it copies a previously tested image from the internal registry to Quay and updates the GitOps repository to trigger deployment.

Verify the Pipeline Configuration

Before running the promotion pipeline, verify that the GitLab repository URL is correct.

```
oc describe pipeline etx-app-promotion -n etx-app-dev
```

Once the pipeline has been created successfully, you're ready to execute a promotion and verify that the application is deployed automatically to the staging cluster.

Lab 16: Test Promotion

In this lab, you'll verify the complete promotion workflow by manually triggering the promotion pipeline and confirming that it copies a container image to Quay and updates the GitOps repository for the staging environment.

By the end of this lab, you will have:

- Selected a container image built by the CI pipeline for promotion
- Triggered the Tekton promotion pipeline
- Verified that the image was copied from the internal registry to Quay
- Confirmed that the GitOps repository was updated with the promoted image

Retrieve an Image Tag

The promotion pipeline requires the tag of an image previously built by the CI pipeline.

List the available image tags:

```
oc get imagestream etx-app-base-app -n etx-app-dev -o json | \
  jq -r '.status.tags[].tag' | head -5
```

Note one of the commit SHA tags from the output. You will use it when creating the PipelineRun.

Refresh the Vault Kubernetes Authentication Token

Before running the promotion pipeline, refresh the Vault Kubernetes authentication configuration.

1. Run the following command:

```
oc exec -n vault etx-vault-0 -- sh -c \
"VAULT_ADDR=\"https://$(oc get route -n vault etx-vault -o
jsonpath='{.spec.host}')\" \
VAULT_TOKEN=\"$(oc get secret -n vault etx-vault-bootstrap -o
jsonpath='{.data.root-token}' | base64 -d)\" \
vault write auth/kubernetes/config \
token_reviewer_jwt=\"$(oc create token etx-vault -n vault --duration=24h)\" \
kubernetes_host='https://kubernetes.default.svc' \
kubernetes_ca_cert=@/var/run/secrets/kubernetes.io/serviceaccount/ca.crt"
```



This refreshes the Kubernetes Token Reviewer JWT used by Vault. In a production environment, this configuration is typically automated and does not require manual intervention.

Trigger the Promotion Pipeline

Create the PipelineRun, replacing `<COMMIT_SHA>` with one of the image tags from the previous step:

```
oc create -f - <<EOF
apiVersion: tekton.dev/v1
kind: PipelineRun
metadata:
  generateName: etx-app-promotion-
  namespace: etx-app-dev
spec:
  pipelineRef:
    name: etx-app-promotion
  params:
    - name: source-image-tag
      value: <COMMIT_SHA>
```

```
    - name: dest-image-tag
      value: stable
    - name: gitops-repo-url
      value: ${GITLAB_URL}/software-factory/etx-app-gitops.git
  workspaces:
    - name: shared-workspace
      persistentVolumeClaim:
        claimName: promotion-workspace-pvc
  taskRunTemplate:
    serviceAccountName: pipeline
EOF
```

Verify the Promotion

Confirm that the promotion pipeline completed successfully.

Watch the PipelineRun status:

```
oc get pipelinerun -n etx-app-dev -w
```

Press `Ctrl` + `C` once the PipelineRun reaches the `Succeeded` state.

The promotion pipeline performs the following actions:

- Retrieves the Quay robot account credentials from Vault
- Copies the selected image from the OpenShift internal registry to Quay
- Updates the staging deployment manifest in the GitOps repository with the promoted image

At this point, the GitOps repository references the promoted image, allowing Argo CD to detect the change and deploy the updated application to the staging environment.

Lab 17: Stage Deployment

In this lab, you'll deploy the staging application by creating an Argo CD Application that targets the secondary cluster. Argo CD will continuously synchronize the staging environment with the manifests stored in the GitOps repository.

By the end of this lab, you will have:

- Created an Argo CD Application for the staging environment
- Configured Argo CD to deploy to the secondary cluster
- Verified that the staging application is synchronized successfully
- Confirmed that the staging deployment is visible in the Argo CD dashboard

Create the Staging Application

In this exercise, you'll create an Argo CD Application that deploys the staging manifests from the GitOps repository to the runtime cluster.

1. Retrieve the GitLab repository URL:

```
GITLAB_URL="https://$(oc get route -n gitlab \
  -l app=webservice \
  -o jsonpath='{.items[0].spec.host}')"

echo "Using GitLab URL: ${GITLAB_URL}"
```

2. Create the Argo CD Application:

```
oc apply -f - <<EOF
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: etx-app-stage
  namespace: etx-app-dev
  labels:
    app.kubernetes.io/part-of: etx_app_base_app
spec:
  project: default
  source:
    repoURL: ${GITLAB_URL}/software-factory/etx-app-gitops.git
    targetRevision: main
    path: environments/stage
  destination:
    name: staging
    namespace: etx-app-staging
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
    syncOptions:
      - CreateNamespace=true
EOF
```



The `staging` destination refers to the secondary cluster that was enrolled with Argo CD in the earlier lab.

Verify the Staging Deployment

Confirm that Argo CD has synchronized the staging application successfully.

1. Check the synchronization status:

```
oc get application etx-app-stage -n etx-app-dev \
  -o jsonpath='{.status.sync.status}{"\n"}'
```

The output should be:

```
Synced
```

2. Open the Argo CD dashboard.
3. Verify that the following applications are listed:
   - `etx-app-dev` – Application deployed to the primary (development) cluster
   - `etx-app-stage` – Application deployed to the secondary (staging) cluster
4. Select the `etx-app-stage` application to review its synchronization status, managed resources, and deployment health.



Because the staging cluster is registered with Argo CD, you can monitor and manage the remote deployment directly from the Argo CD dashboard without logging in to the secondary cluster.

Lab 18: Verify Promotion

In this lab, you'll validate the complete GitOps promotion workflow by verifying that the application is healthy in the development environment and that the promotion pipeline successfully synchronized the staging environment through Argo CD.

By the end of this lab, you will have:

- Reviewed the end-to-end GitOps promotion workflow
- Verified that the development application is healthy
- Confirmed that the GitOps repository references the promoted image
- Verified that Argo CD synchronized the staging application successfully

Understand the Promotion Workflow

In this exercise, you'll review how a container image moves from the development environment to the staging environment using Tekton, Quay, Git, and Argo CD.

The promotion workflow consists of the following steps:

1. The CI pipeline builds the application and pushes a container image to the OpenShift internal registry using the commit SHA as the image tag.
2. The promotion pipeline copies the validated image to the Quay registry and assigns it the `stable` tag.
3. The promotion pipeline updates the `environments/stage/deployment.yaml` manifest in the GitOps

repository with the promoted image.

4. Argo CD detects the Git commit and automatically synchronizes the updated manifests to the staging cluster.

This GitOps workflow provides several advantages:

- **Version control** – All deployment changes are stored and tracked in Git.
- **Auditability** – Every deployment is associated with a Git commit, providing a complete deployment history.
- **Rollback capability** – Reverting a Git commit restores a previous application version.
- **Multi-cluster consistency** – The same Git repository can be used to deploy applications across multiple clusters.

Verify the Development Deployment

Confirm that the development environment is healthy and that the application is running correctly.

Verify the deployment status:

```
# Check Argo CD application status
oc get application etx-app-dev -n etx-app-dev \
  -o jsonpath='{.status.sync.status}: {.status.health.status}{"\n"}'

# Check application pods
oc get pods -n etx-app-dev -l app=etx-app-base-app

# Verify the application health endpoint
curl https://$(oc get route etx-app-base-app -n etx-app-dev \
  -o jsonpath='{.spec.host}')/q/health

# Verify OpenTelemetry auto-instrumentation
oc get pod -n etx-app-dev -l app=etx-app-base-app \
  -o jsonpath='{.items[0].spec.initContainers[*].name}{"\n"}'
```

You should observe:

- Argo CD reports `Synced: Healthy`
- Application pods are in the `Running` state
- The health endpoint returns a successful response
- The `opentelemetry-auto-instrumentation` init container is present



Common Issues

- **ImagePullBackOff** — Verify that the CI pipeline successfully built and pushed the application image.
- **CrashLoopBackOff** — Review the application logs and confirm that required

services such as PostgreSQL and Kafka are available.

- **Argo CD Out of Sync** — Synchronize the application manually from the Argo CD dashboard or verify the GitOps repository changes.
- **Health endpoint unavailable** — Ensure that all supporting services are running and the application has started successfully.

Verify the Staging Promotion

Confirm that the promotion pipeline updated the GitOps repository and that Argo CD synchronized the staging environment.

Verify that the staging deployment manifest references the promoted image. For this, navigate to the clone **etx-app-gitops** repository.

```
cd etx-app-gitops

git pull origin main

grep image: environments/stage/deployment.yaml
```

The deployment manifest should reference the Quay image using the `stable` tag.

Sample Output:

```
image: quay.apps.cluster-f86c6.dyn.redhatworkshops.io/etx-app/etx-app-base-app:stable
```

Verify the staging application status:

```
oc get application etx-app-stage -n etx-app-dev \
  -o jsonpath='{.status.sync.status}{"\n"}'
```

Sample Output:

```
Synced
```

Retreive the ArgoCD application URL and open the Argo CD dashboard.

```
oc get route argocd-server -n etx-app-dev -o jsonpath='https://{.spec.host}{"\n"}'
```

Select the `etx-app-stage` application and verify that:

- The application status is **Synced** and **Healthy**
- The deployment references the promoted image stored in Quay

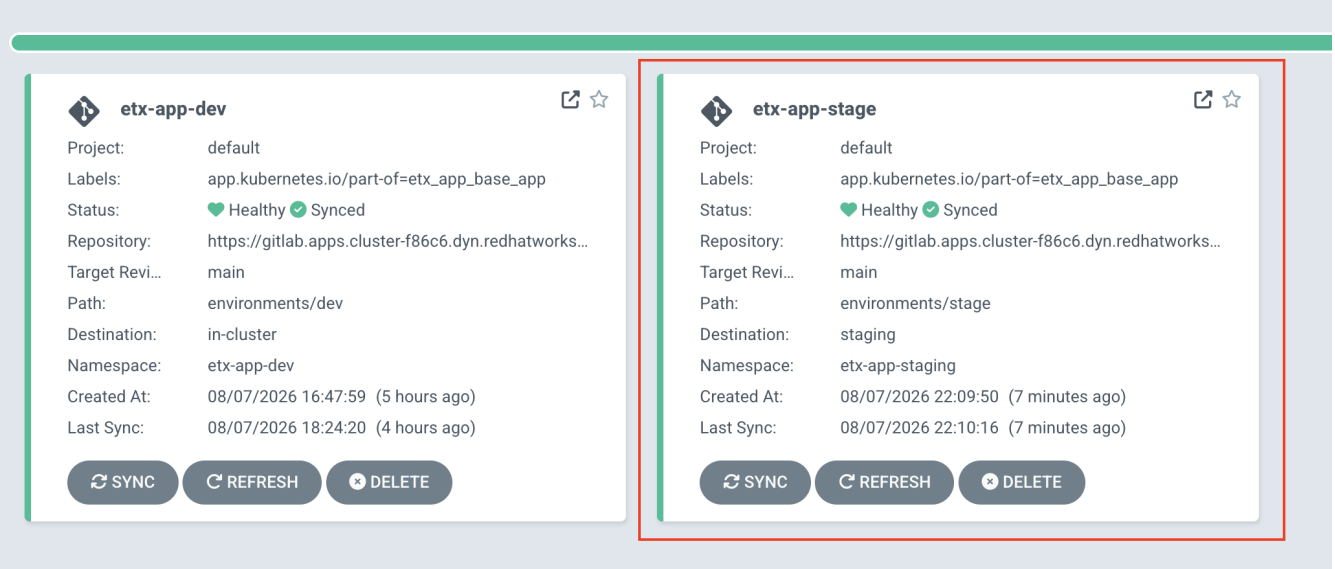


Figure 35. ArgoCD Application status

At this point, you have validated the complete GitOps promotion workflow. A successful promotion updates the GitOps repository, and Argo CD automatically deploys the new application version to the staging cluster.

Lab 19: Promotion Automation (Optional)

In this lab, you'll explore different approaches for controlling application promotion from development to staging, ranging from manual execution to approval-based and Git-driven workflows.

By the end of this lab, you will have:

- Understood different promotion strategies for application releases
- Added an optional manual approval step to a Tekton pipeline
- Learned how GitLab merge requests can be used to control promotions

Understand Promotion Strategies

The promotion pipeline you created in the previous labs can be triggered in several ways depending on your team's release process.

Option 1: Manual Pipeline Execution

The current implementation requires a user to manually start the promotion pipeline after verifying that the application is ready for staging.

This approach is suitable for:

- Small development teams
- Infrequent application releases
- Release processes requiring manual verification

Option 2: Add a Manual Approval Step

Instead of manually starting the pipeline, you can allow the pipeline to start automatically and pause until someone explicitly approves the promotion.

1. Create a Tekton task that waits for manual approval:

```
oc apply -f - <<'EOF'
apiVersion: tekton.dev/v1
kind: Task
metadata:
  name: manual-approval
  namespace: etx-app-dev
spec:
  description: |
    Pauses pipeline execution until manual approval.
    Requires operator intervention to continue.
  steps:
    - name: wait-for-approval
      image: registry.access.redhat.com/ubi9/ubi-minimal:latest
      script: |
        #!/bin/sh
        echo "⏸️  Pipeline paused - waiting for manual approval"
        echo "Run: oc patch taskrun <taskrun-name> -p
'{\"spec\":{\"status\":\"Succeeded\"}}'"
        sleep infinity
EOF
```

   The task pauses the pipeline until an operator manually approves the corresponding TaskRun.

2. Update the promotion pipeline by adding this task as the first task in the pipeline before the image promotion steps.

This allows the pipeline to begin automatically while still requiring explicit approval before promoting an image to the staging environment.

Option 3: Trigger Promotion from GitLab

Another common approach is to promote applications through a Git-based approval workflow.

In this model:

1. Create a dedicated promotion branch in the application repository.
2. Open a Merge Request for the promotion.
3. Request review and approval from the appropriate team members.
4. Configure a GitLab webhook to trigger the promotion pipeline after the Merge Request is merged.

This approach provides several benefits:

- Promotion decisions are reviewed before deployment.
- Git maintains a complete audit history of approvals.
- Teams collaborate using a familiar pull request workflow.
- Promotion becomes part of the standard GitOps development process.

Lab 20: Rollback (Optional)

In this lab, you'll learn two common approaches for rolling back an application deployment in a GitOps environment when a release needs to be reverted.

By the end of this lab, you will have:

- Rolled back a deployment by reverting a Git commit
- Promoted a previously built application image back to staging
- Understood how GitOps enables safe and repeatable rollbacks

Roll Back Using Git

The recommended rollback approach is to restore a previous version of the deployment manifests in the GitOps repository.

1. Navigate to your local **etx-app-gitops** GitOps repository:

```
cd etx-app-gitops
```

2. Review the commit history for the staging deployment manifest:

```
git log environments/stage/deployment.yaml
```

3. Revert the desired commit and push the change:

```
git revert <commit-hash>
git push origin main
```

Once the commit is pushed, Argo CD automatically detects the change and synchronizes the staging environment to the previous application version.

Roll Back by Re-Promoting a Previous Image

If you know the image tag of a previously validated application version, you can rerun the promotion pipeline using that image.

1. Create a new PipelineRun using the previous image tag:

```
oc create -f - <<EOF
apiVersion: tekton.dev/v1
kind: PipelineRun
metadata:
  generateName: etx-app-promotion-
  namespace: etx-app-dev
spec:
  pipelineRef:
    name: etx-app-promotion
  params:
    - name: source-image-tag
      value: <PREVIOUS_SHA>
    - name: dest-image-tag
      value: stable
  workspaces:
    - name: shared-workspace
      persistentVolumeClaim:
        claimName: promotion-workspace-pvc
  taskRunTemplate:
    serviceAccountName: pipeline
EOF
```

The promotion pipeline copies the selected image to Quay, updates the GitOps repository, and Argo CD automatically deploys the previous application version to the staging environment.



In a GitOps workflow, Git remains the source of truth for deployments. Whether you revert a Git commit or re-promote an earlier application image, Argo CD continuously reconciles the cluster to match the desired state stored in the GitOps repository.

Lab Summary

Lab Summary

You have completed the Promotion Pipeline exercise and established a secure, GitOps-based application promotion workflow.

You now have the following capabilities:

- **Automated Image Promotion** – Application images can be promoted from the internal registry to Quay without rebuilding.
- **GitOps-Based Deployments** – Staging deployments are managed through Git repositories and synchronized by Argo CD.
- **Secure Credential Management** – Registry credentials are securely retrieved and managed using Vault.
- **Multi-Cluster Deployment Support** – Application versions can be promoted across different

OpenShift environments.

- **Rollback Capability** – Previous application versions can be restored using Git history or previously validated images.

Key Takeaways

Promotion Does Not Mean Re-Build

The promotion process moves the same validated container image between environments.

- The same container image identified by its SHA is promoted from development to staging.
- No application rebuild occurs between environments.
- Configuration changes are applied separately from the application artifact.
- This ensures that the application you test is exactly the application you deploy.

GitOps Ensures Consistency

Git remains the source of truth for application deployments.

- All deployment state is stored and versioned in Git.
- Changes are fully auditable and reversible.
- Argo CD continuously synchronizes the cluster state with the desired state defined in Git.
- Drift detection ensures that environments remain consistent.

Multi-Cluster Architecture

The promotion workflow supports application delivery across multiple OpenShift environments.

- Development uses the internal registry for fast local image builds and testing.
- Staging uses Quay for a trusted, externally accessible image registry.
- Argo CD provides centralized deployment management across clusters.

Knowledge Check

<iframe type="text/javascript" src="_attachments/questions.html" style="width: 100%; max-width: 1024px; height: 800px" allowfullscreen webkitallowfullscreen mozAllowFullScreen allow="autoplay *; fullscreen *; encrypted-media *" frameborder="0"></iframe>

Summary and What's next?

Congratulations! You have successfully built an automated, GitOps-driven application delivery workflow.

In this module, you learned how **OpenShift Pipelines** automates Continuous Integration, how **OpenShift GitOps and Argo CD** provide continuous delivery through Git-based desired state, and how applications can be securely promoted across environments without rebuilding.

You created and automated a CI pipeline, connected it to a GitOps deployment workflow, configured secure authentication and credential management, and implemented application promotion using **OpenShift Pipelines, Quay, Vault, and Argo CD**. You also explored how Git history and validated images support reliable rollback and recovery.

You now have a complete foundation for building, testing, deploying, promoting, and managing applications through a standardized Software Factory workflow.

In the next modules, you'll build on this foundation to implement **developer self-service, application modernization, and trusted software supply chain practices**.